UNIVERZITET U BEOGRADU

ELEKTROTEHNIČKI FAKULTET



REALIZACIJA RAG SISTEMA I EVALUACIJA PRETRAGE NAD DOKUMENTACIJOM PROGRAMSKOG JEZIKA PYTHON

Diplomski rad

Mentor:

prof. dr Marko Mišić

Kandidat:

Nemanja Zelic

0092/2022

Beograd, septembar 2026.

SADRŽAJ

1 UVOD **4**
2 PROBLEM I TEORIJSKE OSNOVE **6**
2.1 Generisanje odgovora na osnovu dodatnog konteksta 6
2.2 Leksička pretraga i BM25 7
2.3 Vektorsko predstavljanje teksta 8
2.4 Spajanje rang-lista 9
2.5 Segmentacija i granice konteksta 10
2.6 Postojeća rešenja i predmet istraživanja 12
3 KORPUS I KORIŠĆENE TEHNOLOGIJE **13**
3.1 Izbor i obuhvat korpusa 13
3.2 Obrada RST sadržaja 14
3.3 Tri varijante strukturnih segmenata 16
3.4 Referentni indeks i poreklo artefakata 18
3.5 Evaluacioni skup 19
3.6 Modeli i softversko okruženje 19
4 REALIZACIJA SISTEMA **22**
4.1 Arhitektura i tok podataka 22
4.2 Identitet i pristup segmentima 24
4.3 Obrada nepoznatih direktiva 24
4.4 Ograničenje kontrole dužine segmenata 25
4.5 Spajanje nezavisno od skale skorova 26
4.6 Izvršavanje modela i upravljanje memorijom 26
4.7 Pravilo odbijanja i generisanje 27
5 METODOLOGIJA ANALIZE I TESTIRANJA **29**
5.1 Eksperimentalni plan 29
5.2 Relevantnost i položaj rezultata 30
5.3 Metrike uspešnosti pretrage 30
5.4 Merenje latencije i veličine indeksa 31
5.5 Procena pravila odbijanja 32
5.6 Funkcionalna provera 32
5.7 Ponovljivost i ograničenja eksperimentalnog postupka 33
5.8 Postupak analize odgovora u studiji slučaja 33
6 REZULTATI I DISKUSIJA **34**
6.1 Referentne metode pretrage 34
6.2 Analiza pojedinačnih pitanja 35
6.3 Uticaj konfiguracije strukturnih segmenata 36
6.4 Broj segmenata i veličina indeksa 39
6.5 Latencija pretrage 40
6.6 Odbijanje pitanja 41

6.7 Ograničenja i validnost rezultata 42
7 OPIS RADA SISTEMA I STUDIJA SLUČAJA 43
7.1 Priprema i osnovne komande 43
7.2 Tumačenje JSON odgovora 44
7.3 Upit o preporuci PEP 8 44
7.4 Pitanje izvan korpusa 45
7.5 Koristan odgovor iz neoznačenog dokumenta 46
7.6 Pogrešan odgovor uz dovoljan kontekst 46
7.7 Tačan odgovor uprkos lošijem dokumentnom rangu 47
7.8 Praktična ograničenja 47
8 ZAKLJUČAK 48
LITERATURA 50
SPISAK SKRAĆENICA 52
SPISAK SLIKA 53
SPISAK TABELA 54
PRILOG A EVALUACIONA PITANJA 55
PRILOG B IDENTIFIKATORI IZVORA I MODELA 58
PRILOG C UPIT ZA GENERISANJE ODGOVORA 59

1 UVOD

Tehnička dokumentacija programskog jezika Python obuhvata objašnjenja jezičkih konstrukcija, opise standardne biblioteke i predloge izmena jezika. Odgovor na konkretno pitanje često je sažet, dok se u dokumentu koji ga sadrži obrađuju brojne povezane teme. Korisnik zato mora da prepozna odgovarajući dokument, pronađe relevantan odeljak i protumači opis u kontekstu sopstvenog problema. Pretraga po ključnim rečima može biti efikasna kada pitanje sadrži tačan naziv funkcije ili klase, ali situacije u kojima korisnik opisuje ponašanje programa, a ne zna termin kojim je ono predstavljeno u dokumentaciji, predstavljaju znatno teži slučaj.

Generisanje potpomognuto pretragom, poznato pod nazivom *Retrieval-Augmented Generation* (RAG), povezuje pronalaženje izvora sa jezičkim modelom koji oblikuje odgovor. Lewis i saradnici opisali su pristup u kome generativni model koristi i znanje zapisano u parametrima i sadržaj pronađen u spoljašnjem indeksu [1]. U ovom radu taj princip primenjen je kao programski tok sa odvojenim fazama pripreme korpusa, pretrage i generisanja. Modeli se koriste sa prethodno obučenim parametrima, bez dodatnog obučavanja na izabranom korpusu.

Predmet rada je realizacija lokalnog RAG sistema nad dokumentacijom CPythona i *Python Enhancement Proposal* (PEP) dokumentima. U eksperimentalnom delu upoređuju se vektorska pretraga, leksička pretraga BM25 i njihovo spajanje postupkom *Reciprocal Rank Fusion*, uz različitu granularnost strukturnih segmenata. Jedinica pretrage je segment, dok postojeće evaluacione oznake određuju relevantan dokument.

Cilj rada je da se ispita kako izbor metode pretrage i načina podele dokumenata utiče na pronalaženje relevantnog sadržaja i utrošak računarskih resursa. Poređenje je organizovano oko sledećih istraživačkih pitanja:

RQ1: Kako spajanje rezultata vektorske i leksičke pretrage utiče na rangiranje u odnosu na pojedinačne metode, naročito na prva dva rezultata koja koristi generator?

RQ2: Koja od posmatranih konfiguracija strukturne segmentacije ostvaruje najveći *MRR* na izabranom skupu pitanja?

RQ3: Kako se izbor te konfiguracije odražava na broj segmenata, prostor potreban za indeks i vreme pretrage?

Za ispitivanje ovih pitanja izabrana je dokumentacija programskog jezika Python, u kojoj se objašnjenja u prirodnom jeziku smenjuju sa potpisima funkcija, primerima programa i tabelama. Takva struktura omogućava da se razmotri kako način obrade izvora utiče na tekst dostupan pretraživaču. Sa korisničke strane, pitanja mogu sadržati tačna imena programskih elemenata ili opisivati njihovu namenu, pa je isti korpus pogodan za poređenje leksičkog i vektorskog pristupa.

Drugo poglavlje uvodi problem, metode pretrage i povezane radove. Treće opisuje korpus, segmentaciju, pitanja i modele. Četvrto prikazuje arhitekturu i probleme implementacije. U petom su definisani protokol i metrike, a u šestom predstavljeni rezultati i analiza grešaka. Sedmo poglavlje opisuje korišćenje aplikacije i studiju slučaja. Zaključak odgovara na istraživačka pitanja i određuje naredne korake validacije i razvoja.

2 PROBLEM I TEORIJSKE OSNOVE

Teorijske osnove uvode se objašnjenjem kako se pronađeni dokumenti koriste pri generisanju odgovora i zbog čega se kvalitet pretrage posmatra odvojeno od tačnosti odgovora. Izložene su prema radu o RAG pristupu [1], a u nastavku su, uz odgovarajuće izvore, opisani BM25, vektorsko predstavljanje teksta i spajanje rang-lista. Završni deo povezuje ove postupke sa predmetom eksperimentalnog istraživanja u radu.

2.1 Generisanje odgovora na osnovu dodatnog konteksta

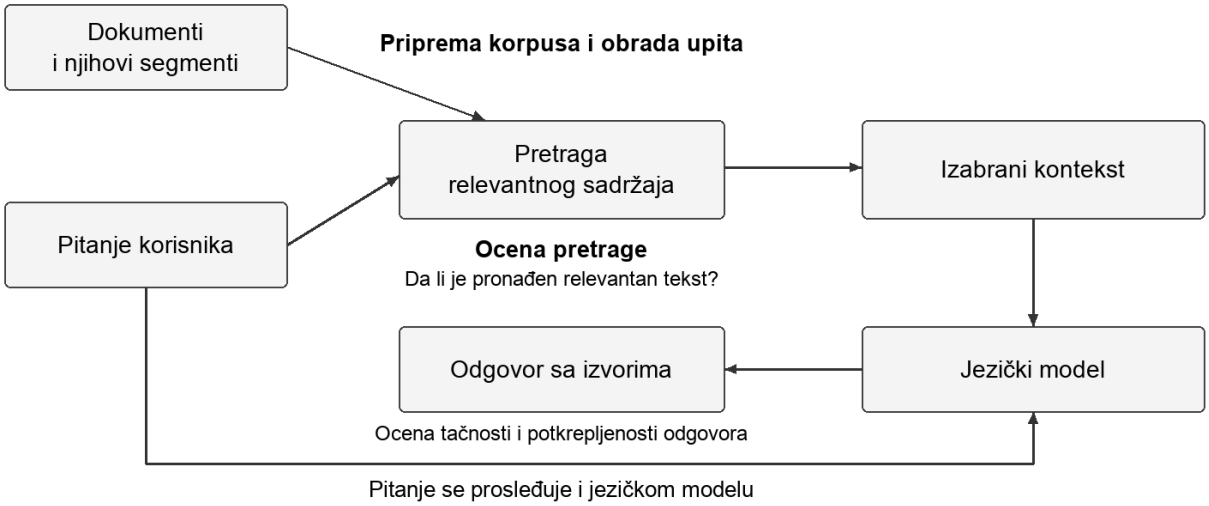
Sistem za odgovaranje nad dokumentacijom prima pitanje u prirodnom jeziku, pronalazi tekst koji može da posluži kao osnova i proizvodi sažet odgovor sa vezom ka izvoru. Tekst prosleđen jezičkom modelu naziva se kontekstom za generisanje, a njegova korisnost zavisi od toga da li objašnjava traženo ponašanje. Na primer, pitanje o tome da li kontekstni menadžer veze *sqlite3* automatski zatvara vezu zahteva razlikovanje upravljanja transakcijom od zatvaranja resursa.

U posmatranom rešenju jedinica indeksiranja je segment dokumenta, pri čemu svaki segment nosi identitet izvornog dokumenta i tekst koji se može proslediti generatoru. Dokument i segment zato imaju različite uloge: dokument je jedinica označavanja relevantnosti u evaluaciji, a segment je jedinica koju pretraživač rangira. Dva visoko rangirana segmenta mogu poticati iz istog dokumenta – takav rezultat može biti koristan ako sadrže različite potrebne informacije, ali može i da zauzme ograničeni kontekst ponovljenim sadržajem.

RAG tok može da zakaže na više nezavisnih mesta: odgovor možda nije prisutan u preuzetom korpusu, parser može da izgubi važan tekst, podela može da razdvoji tvrdnju od uslova pod kojima važi, pretraživač može da rangira tematski blizak ali nedovoljan segment, a generator može pogrešno da protumači ispravno pronađen sadržaj. Zbog toga se kvalitet pretrage ne poistovećuje sa kvalitetom konačnog odgovora.

Slika 2.1.1 prikazuje ove faze i mesta na kojima se proverava uspešnost sistema. Korpus se priprema pre postavljanja pitanja, dok pretraživač pri svakom upitu bira samo deo njegovog sadržaja. Generator zatim dobija pitanje i izabrani kontekst i ne pristupa samostalno svim

dokumentima u indeksu. Slika je autorska šema osnovnog postupka, zasnovana na principu opisanom u [1].



Slika 2.1.1 Osnovne faze generisanja odgovora na osnovu pronađenog konteksta

2.2 Leksička pretraga i BM25

Leksička pretraga koristi pojavljivanje termina iz upita u indeksiranom tekstu. BM25 povezuje učestalost termina, njegovu rasprostranjenost u kolekciji i dužinu dokumenta [2]. Doprinos učestalosti termina zasićuje se, pa svako novo pojavljivanje sve manje povećava skor, dok se doprinos termina koji se pojavljuje u velikom delu kolekcije umanjuje. Normalizacija dužine sprečava da duži tekst bude favorizovan samo zato što sadrži više reči.

Kod tehničke dokumentacije leksičko poklapanje ima jasnu praktičnu vrednost. Nazivi kao što su *FrozenInstanceError*, *lru_cache* i *add_subparsers* imaju precizno značenje i predstavljaju dobre signale relevantnosti. S druge strane, pitanje o svrsi neke konstrukcije može koristiti drugačije reči od dokumentacije. U takvim slučajevima leksički signal može biti slab iako odgovarajući tekst postoji.

Implementacija koristi SQLite FTS5, koji pruža indeks punog teksta, tokenizaciju i funkciju *bm25*. SQLite rangira bolje rezultate manjom numeričkom vrednošću svoje BM25 funkcije, pa implementacija pri čitanju rezultata menja znak radi jedinstvenog prikaza skora [3]. To ne menja redosled SQL rezultata. Težine tekstualnih kolona omogućavaju da naslov ili putanja sekcije imaju drugačiji uticaj od osnovnog teksta.

Izbor tokenizacije je deo sistema pretrage. U projektu se koristi kombinacija Porter i Unicode61 tokenizacije. Upit se dodatno normalizuje, a izrazi sa slovima i ciframa proširuju se sastavnim delovima. Tako PEP8 daje i termine pep i 8. Ovo povećava šansu za poklapanje sa zapisom PEP 8, ali istovremeno može proširiti skup kandidata. Upotreba operatora OR čini pretragu tolerantnom na delimično poklapanje, uz potrebu da rangiranje izdvoji najbolje rezultate.

U BM25 modelu dokument d označava jedan indeksirani segment. Opšti oblik skora za upit q prikazan je jednačinom 2.2.1 [2]. Učestalost termina t u segmentu je f(t,d), dužina segmenta |d|, a prosečna dužina kolekcije avgdl. Faktor IDF(t) umanjuje doprinos termina čestih u kolekciji. Parametar k_1 upravlja zasićenjem učestalosti, dok b određuje jačinu normalizacije dužine.

$$BM25(q,d) = \sum_{t} \in q\ IDF(t) \cdot \frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1 \cdot (1 - b + b \cdot |d|/avgdl)} \quad (2.2.1)$$

SQLite FTS5 koristi $k_1 = 1{,}2$ i $b = 0{,}75$ i menja znak ukupne vrednosti, tako da se bolji rezultati dobijaju rastućim sortiranjem. Učestalost se može ponderisati po kolonama, a negativan IDF zamenjuje se malom pozitivnom vrednošću [3]. Zato treba razlikovati opštu BM25 formulu od konkretne FTS5 realizacije. Za dva segmenta jednake dužine i isti termin, rast f sa jedan na dva povećava doprinos manje nego dvostruko: deo bez IDF raste sa 1 na približno 1,375 kada je |d| = avgdl. Time se ublažava prednost ponavljanja istog termina.

2.3 Vektorsko predstavljanje teksta

Vektorska pretraga preslikava tekst pitanja i segmenata u vektore iste dimenzije, a sličnost se zatim računa u tom prostoru. U ovom radu dimenzija je 512. Izlazni vektori normalizuju se tako da imaju jediničnu euklidsku normu, pa je skalarni proizvod dva tako normalizovana vektora jednak njihovoj kosinusnoj sličnosti.

Model Jina Embeddings 2 pripada porodici modela namenjenih predstavljanju teksta vektorima [4]. U projektu je izabrana varijanta jina-*embeddings*-v2-small-en, čija kartica opisuje model za engleski tekst i vektore sa 512 komponenti [5]. Izbor odgovara jeziku korpusa i pitanja. Time nije potvrđena ista uspešnost za pitanja na srpskom, iako generativna instrukcija traži odgovor na jeziku pitanja.

U odnosu na leksički pristup, vektorska pretraga može povezati različite formulacije sličnog značenja. Međutim, visoka sličnost ne predstavlja dokaz da segment sadrži odgovor – dva teksta mogu govoriti o istoj biblioteci, a opisivati različite operacije. Skor modela zato se koristi za rangiranje i heurističko odbijanje, bez tumačenja kao kalibrisane verovatnoće tačnog odgovora.

Pretraga vektora realizovana je pomoću biblioteke FAISS [6]. Konkretno se koristi *IndexFlatIP*, koji računa skalarni proizvod nad svim vektorima u indeksu. Pretraga ovim indeksom je egzaktna i ne uvodi grešku aproksimativnog izbora suseda. To pojednostavljuje tumačenje eksperimenta, jer promene rezultata nisu posledica podešavanja aproksimativnog indeksa. Cena je rast obima računanja sa brojem vektora.

Za vektore u i v kosinusna sličnost definisana je jednačinom 2.3.1. L2 normalizacija svodi imenilac na jedan, pa *IndexFlatIP* rangira prema istom kriterijumu. Nulti vektor ne bi imao definisanu kosinusnu sličnost, pa normalizacija pretpostavlja nenulti izlaz modela.

$$cos(u, v) = \frac{u \cdot v}{\|u\|_2 \cdot \|v\|_2} \tag{2.3.1}$$

Enkoder najpre tokenizuje tekst, a slojevi pažnje formiraju kontekstualno predstavljanje svakog tokena. Objedinjavanje tih predstava daje jedan vektor za ceo segment. Upit i tekst korpusa prolaze kroz isti model, pa pretraga poredi njihove izlaze u zajedničkom prostoru. Naslov odeljka u ulazu utiče na sve naredne korake ovog predstavljanja, a ne predstavlja zaseban faktor FAISS skora.

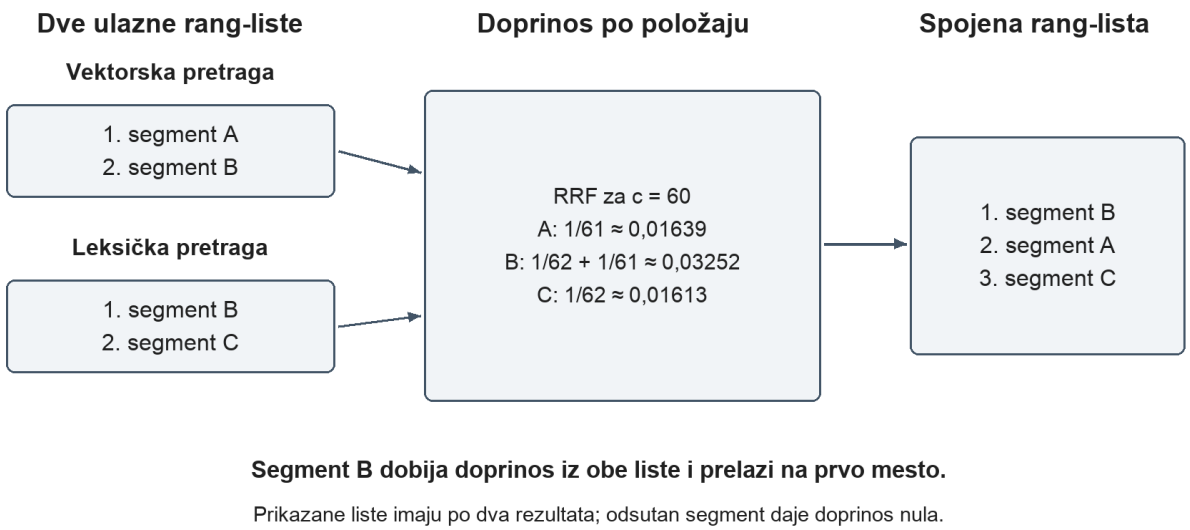
2.4 Spajanje rang-lista

Vektorski i BM25 skorovi nisu izraženi na istoj skali. Direktno sabiranje zahtevalo bi dodatnu normalizaciju i izbor relativnih težina. *Reciprocal Rank Fusion*, skraćeno RRF, umesto sirovih skorova koristi položaje rezultata na rang-listama [7]. Svaki rezultat dobija zbir doprinosa iz lista na kojima se pojavljuje.

U posmatranoj implementaciji doprinos rezultata na poziciji r iznosi 1/(60 + r), pri čemu rang počinje od jedan. Segment koji se nalazi na obe liste dobija dva doprinosa. Spajanje se obavlja po identifikatoru segmenta, a ne po identifikatoru dokumenta. Rezultat može napredovati zahvaljujući saglasnosti dve metode čak i kada ni na jednoj pojedinačno nije prvi.

Na jednostavnom primeru, ako vektorska lista počinje segmentima A i B, a leksička segmentima B i C, segment B dobija zbir 1/62 + 1/61. Segment A dobija samo 1/61, a C samo 1/62. B tada postaje prvi na spojenoj listi. Ovaj primer objašnjava postupak, a njegova korisnost u sistemu utvrđuje se rezultatima eksperimenata.

Slika 2.4.1 prati ovaj primer od dve ulazne liste do konačnog poretka. Računaju se položaji segmenata, pa razlika u skalama njihovih izvornih skorova ne utiče na spajanje. Autorska šema ilustruje postupak RRF [7] na listama sa po dva segmenta.



Slika 2.4.1 Spajanje dve rang-liste pomoću RRF postupka

Spajanje ne garantuje poboljšanje za svako pitanje. Dobar rezultat jedne metode može pasti ako druga metoda podigne više drugih segmenata. Takođe, saglasnost može nastati oko teksta koji je samo tematski blizak. U šestom poglavlju prikazani su i takvi slučajevi, jer prosečna vrednost sama ne otkriva raspodelu dobitaka i gubitaka.

Opšta suma u jednačini 2.4.1 obuhvata liste L na kojima se segment s pojavljuje. $r_l(s)$ označava njegov rang u listi l, a c konstantu koja ublažava prednost prvih pozicija. Za odsutan segment doprinos odgovarajuće liste je nula. U aplikaciji su dve liste i c = 60 [7].

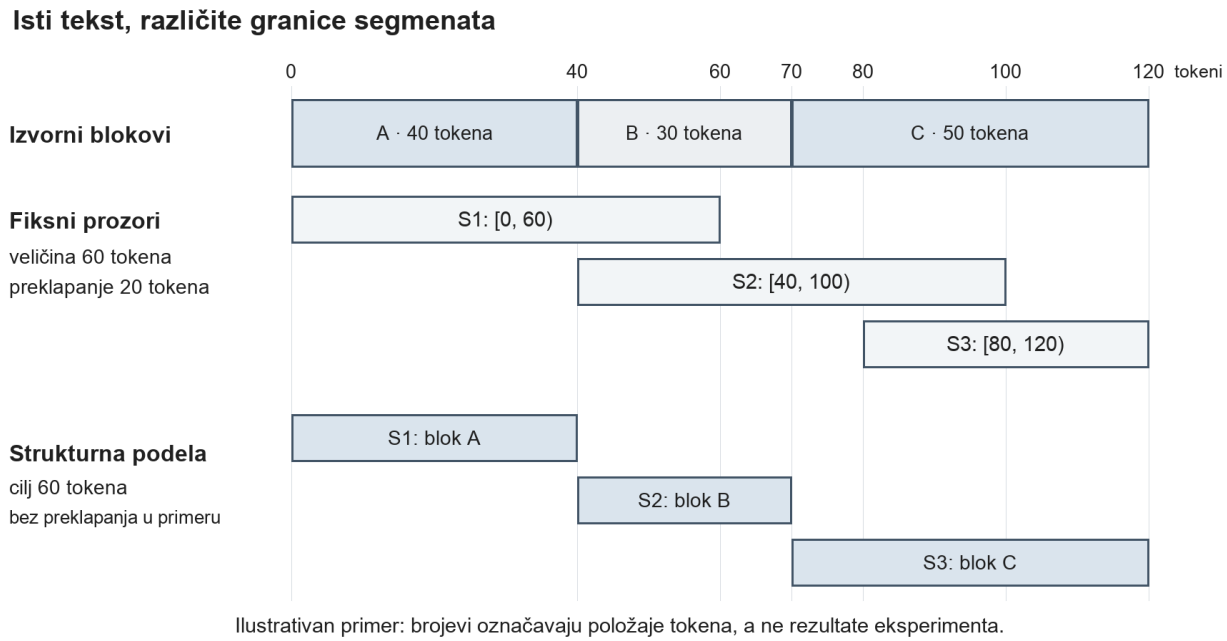
$$RRF(s) = \sum_{l \in L} \frac{1}{c + r_l(s)} \tag{2.4.1}$$

2.5 Segmentacija i granice konteksta

Dugi dokumenti dele se na segmente kako bi pretraga izdvojila uži tekst i kako bi generator dobio ograničenu količinu konteksta. Token je jedinica na koju tokenizator razlaže tekst: može predstavljati reč, deo reči, znak ili drugi niz znakova. Broj tokena zato nije isto što i broj reči. Fiksni prozor deli tok tokena prema zadatoj veličini i koraku. Granica može pasti usred objašnjenja, liste ili primera koda, pa preklapanje susednih prozora ublažava gubitak neposrednog konteksta uz povećanje obima indeksa.

Strukturna segmentacija najpre prepoznaje odeljke i blokove, a zatim ih grupiše. U ovoj realizaciji cilj je da se ne spajaju različite sekcije i da se kratki blokovi zadrže celi. Ako je pojedinačni blok prevelik, ipak se koristi podela na tokene. Time se dobija kompromis između semantičkih granica dokumenta i ograničenja veličine.

Autorska šema na slici 2.5.1 poredi prozore od 60 tokena, sa preklapanjem od 20 tokena, i podelu prema blokovima. Prozori seku blokove B i C, dok ih strukturna podela zadržava cele jer nijedan par susednih blokova ne staje u zadati cilj. Strukturni primer je radi preglednosti bez preklapanja, a stvarni prenos završnih blokova opisan je u odeljku 4.4. Dužine su ilustrativne i ne predstavljaju merenja korpusa.



Slika 2.5.1 Fiksni prozori i podela prema granicama blokova

Manji segment može da izdvoji relevantnu tvrdnju i smanji uticaj okolnih tema na vektor, ali može i da izgubi neophodan uslov ili definiciju iz prethodnog pasusa. Veći segment zadržava više konteksta, ali može povezati više različitih informacija u jedan vektor. Ova suprotstavljena dejstva obrazlažu potrebu za eksperimentom, jer sama veličina segmenta ne određuje unapred ishod.

Broj tokena zavisi od tokenizatora. Tokeni kojima se upravlja segmentacijom u ovom projektu potiču iz *cl100k_base* kodiranja biblioteke *tiktoken* [8]. Model za vektorsko predstavljanje koristi sopstveni tokenizator. Zato segment sa 256 *cl100k_base* tokena ne mora imati 256 tokena na ulazu u Jina model. Dodatni naslov i putanja sekcije takođe povećavaju tekst koji model za vektorsko predstavljanje obrađuje.

2.6 Postojeća rešenja i predmet istraživanja

Izvorni RAG rad [1] uvodi modele koji zajednički koriste parametarsku i neparametarsku memoriju. Ovde se preuzima osnovna ideja oslanjanja na pronađeni tekst, dok realizacija ostaje modularna: indeksiranje, pretraga i generisanje odvojeni su programski koraci. Nisu realizovani zajedničko obučavanje *retrievera* i generatora niti originalne formulacije *RAG-Sequence* i *RAG-Token*.

BEIR pokazuje značaj evaluacije pretrage na različitim domenima i uključuje leksičke, vektorske i postupke dodatnog rangiranja [9]. Za ovaj rad je relevantna metodološka pouka da se uspešnost jedne metode proverava na zadatku, umesto da se pretpostavlja na osnovu arhitekture. Ovde nije korišćen BEIR skup niti su rezultati neposredno uporedivi sa njegovim vrednostima.

Generisanje se oslanja na model iz porodice Qwen2.5, opisane u tehničkom izveštaju [10]. Korišćena je mala instrukciona varijanta Qwen2.5-0.5B-Instruct [11]. Ona omogućava da aplikacija, nakon pretrage, generiše odgovor u lokalnom okruženju. Njene osobine i način izvršavanja opisani su u odeljku 3.6.

Postojeće metode u radu predstavljaju sastavne delove rešenja i eksperimentalne osnove za poređenje. Poređenje sa gotovim RAG proizvodom zahtevalo bi definisanje njegove verzije, modela, korpusa, načina segmentacije i dostupnih izlaza za evaluaciju. Takvo poređenje nije deo izvedenog eksperimenta. Dobijeni rezultati odnose se na tri metode pretrage u realizovanom sistemu.

Metodologija Ragas razdvaja kvalitet pronađenog konteksta, vernost odgovora kontekstu i relevantnost odgovora za pitanje [12]. Ta podela pomaže da se odredi domet ovde izmerenih metrika: položaj označenog dokumenta daje informaciju o pretrazi, dok bi ocena potkrepljenosti zahtevala čitanje stvarnih segmenata i tvrdnji odgovora. Ragas nije pokretan u ovom eksperimentu. Njegove dimenzije služe za razgraničenje postojećih rezultata i potrebne dopune evaluacije.

Za Python dokumentaciju zato se porede leksička pretraga BM25, vektorska pretraga i njihova RRF fuzija. Njihova uspešnost proverava se na istim pitanjima, dok se tačnost i potkrepljenost generisanih odgovora razmatraju odvojeno.

Eksperiment poredi tri konfiguracije segmentacije uz iste modele i metode. Veličina segmenata i preklapanje menjaju se zajedno, pa njihov zaseban uticaj nije ispitan.

3 KORPUS I KORIŠĆENE TEHNOLOGIJE

Osnovu eksperimenta čine dokumenti nad kojima sistem radi, njihova priprema za indeksiranje i skup pitanja korišćen pri evaluaciji. Opis izvora zasniva se na dokumentaciji i repozitorijumima CPython i PEP projekta [13, 14], a postupak parsiranja na Docutils dokumentaciji [15]. Zatim su predstavljeni modeli i programsko okruženje, uz razdvajanje izabranih parametara od svojstava stvarno sačuvanih podataka.

3.1 Izbor i obuhvat korpusa

Korpus čine izvori zvanične dokumentacije CPythona iz direktorijuma Doc i PEP dokumenti iz odgovarajućeg repozitorijuma Python Software Foundation [13, 14]. U dokumentaciji se pretražuju datoteke sa nastavcima rst i txt, a u PEP repozitorijumu datoteke rst. Izabrane su konkretne revizije repozitorijuma, pa sadržaj eksperimenta ne zavisi od naknadnih promena glavne razvojne grane.

Dokumentacija CPythona obuhvata 561 datoteku i 13.357.426 bajtova, a PEP deo 755 datoteka i 14.481.006 bajtova. Ukupno je obrađeno 1.316 datoteka i 27.838.432 bajta. Ove veličine predstavljaju izvorne datoteke, pre parsiranja, normalizacije i segmentacije. One ne predstavljaju veličinu indeksa niti broj tokena koji ulaze u modele.

Tabela 3.1.1 Obuhvat izvornog korpusa

Izvor	Datoteke	Bajtovi
CPython Doc	561	13.357.426
Python PEP	755	14.481.006
Ukupno	1.316	27.838.432

Korpus je preuzet iz tačno određenih stanja dva izvorna repozitorijuma. Git svako takvo stanje označava identifikatorom izmene, odnosno *commit*-a. Taj identifikator omogućava ponovno pronalaženje istih datoteka, dok broj poput Python 3.14 označava izdanje jezika. Veze ka korišćenim stanjima korpusa i modela izdvojene su u prilog B, zajedno sa njihovom namenom.

Sačuvani izvori obuhvataju dokumentaciju različitih delova jezika, istorijske preglede izdanja i radnu datoteku „What's new in Python 3.16“. Prisustvo te datoteke pokazuje da korpus

uključuje i razvojni sadržaj, ali ne potvrđuje da svi njegovi delovi opisuju jedno završeno izdanje. Verzija Python interpretera korišćenog za izvršavanje aplikacije zaseban je podatak, naveden u odeljku 3.6.

Dokumentacija i PEP predlozi nemaju potpuno istu namenu. Dokumentacija opisuje jezik i njegovu upotrebu, dok PEP može obrazlagati promenu, alternative i odluke iz procesa njenog razvoja. Pretraživač ih sme povezati tematski, ali generator ne bi trebalo automatski da svaki pronađeni predlog predstavi kao ponašanje svih verzija Pythona. U sadašnjoj realizaciji nema posebnog filtera po statusu PEP-a ili verziji jezika. To ostaje ograničenje primene na pitanja koja zahtevaju takvu preciznost.

Broj ulaznih datoteka treba razlikovati od zastupljenosti u indeksu. Sve tri strukturne varijante imaju segmente iz 1.315 dokumenata. Datoteka *cpython-docs/deprecations/index.rst* sadrži naslove i include direktive, ali ne daje tekstualni segment u ovoj obradi. Uključene datoteke postoje kao zasebni izvori, pa izostanak ovog zbirnog dokumenta ne dokazuje da je izgubljen i sav tekst na koji on upućuje.

Primer je PEP 703, koji opisuje opcionalnu izgradnju CPythona bez globalne brave interpretera, odnosno GIL-a. U preuzetoj datoteci ima *status Final* i polje *Python-Version* sa vrednošću 3.13. Ti podaci ne znače da je svaka instalacija Pythona bez GIL-a. Odgovor zato mora uzeti u obzir *status* dokumenta, verziju i način izgradnje. Indeks čuva i istorijske i razvojne izvore bez filtriranja po tim osobinama.

3.2 Obrada RST sadržaja

Izvorni dokumenti koriste format *reStructuredText* (RST). Docutils obezbeđuje parser koji tekst prevodi u stablo dokumenta, sa sekcijama i drugim elementima [15]. Python dokumentacija koristi i dodatne direktive i uloge. Ako se dokument obrađuje kao običan tekst, bez tumačenja oznaka njegove strukture, u rezultatu mogu ostati nepotrebne oznake, a veze između naslova i pripadajućih odlomaka mogu se izgubiti.

Eksperimentalna obrada najpre pronalazi nazive direktiva i uloga u celom korpusu. Za one koje parser ne poznaje registruje opšti način očuvanja sadržaja. Direktiva se pretvara u kontejner sa njenim argumentom i obrađenim unutrašnjim tekstom, dok se nepoznata uloga čuva kao literal. To omogućava da tekst ostane dostupan pretrazi, bez tvrdnje da je reprodukovana puna semantika svih Sphinx proširenja.

Pri obilasku stabla čuvaju se pasusi, blokovi koda, liste, tabele i srodni tekstualni elementi. Naslovi sekcija grade putanju od opštijeg ka užem odeljku. Komentari, sistemske poruke parsera i sirovi elementi ne uključuju se u pretraživi tekst. Učitavanje spoljnih datoteka i obrada raw sadržaja isključeni su parametrima parsera. Takva podešavanja ograničavaju šta parser izvršava ili uključuje tokom obrade izvora.

Naslov PEP dokumenta izdvojen je iz polja *Title* kada ono postoji. U drugim dokumentima koristi se prvi raspoloživi naslov sekcije ili naziv datoteke. Ovakva pravila su praktična, ali nisu potpuna rekonstrukcija objavljene HTML dokumentacije. Posebno treba razlikovati očuvani tekst tabele od njenog originalnog vizuelnog rasporeda, jer se za indeksiranje koristi tekstualni prikaz čvora.

Dopunska provera parsera nad svih 1.316 izvora koristi isti kod i podešavanja. Jedini dokument bez izdvojenih blokova je *deprecations/index.rst*. U stablima su prebrojane 1.693 sistemske poruke nivoa INFO, 2.224 nivoa WARNING i 1.632 nivoa ERROR. Broj poruka nije broj neispravnih dokumenata: jedan izvor može dati više poruka. Prvobitna obrada izostavlja te čvorove iz teksta, pa uspešan završetak parsiranja sam ne potvrđuje očuvanje svih informacija.

Tabela 3.2.1 prikazuje šest malih kontrolnih RST primera obrađenih istom funkcijom. Primeri su namenski sastavljeni za proveru konstrukcija i nisu predstavljeni kao odlomci iz korpusa ili dodatna evaluaciona pitanja. Potpis funkcije i uvučen kod ostaju čitljivi. Tabela se svodi na niz tekstova ćelija, include ne učitava spoljnu datoteku, a unakrsna veza zadržava oznaku bez razrešene destinacije.

Tabela 3.2.1 Kontrolni primeri obrade RST konstrukcija

Konstrukcija	Izvorni RST	Izdvojeni tekst

Potpis funkcije	`.. function:: sample(x, *, flag=False)`
	
	`   Return x when flag is false.`
		`sample(x, *, flag=False)`
		
		`Return x when flag is false.`

Tabela	`+-------+-------+`
	`| Name  | Value |`
	`+=======+=======+`
	`| alpha | 1     |`
	`+-------+-------+`
		`Name`
		
		`Value`
		
		`alpha`
		
		`1`

Lista	`* first item`
	`* second item`
		`first item`
		
		`second item`

Kod	`Example::`
	
	`    for x in items:`
	`        print(x)`
		`Example:`
		
		`for x in items:`
		`    print(x)`

Include	`.. include:: fragment.rst`	`(prazno)`
Unakrsna veza	`See :func:`sample` and :ref:`target-label`.`	`See sample and target-label.`

Ovakvi gubici imaju različite posledice. Linearizovana tabela može razdvojiti vrednost od naziva kolone, dok nerazrešena veza ostavlja termin bez objašnjenja na koje upućuje. Za postojeća pitanja ne postoji potpuna anotacija dokaznih raspona koja bi kvantifikovala posledicu svake poruke parsera. Prebrojavanje praznih rezultata i kontrolni primeri zato proveravaju obradu, ali ne zamenjuju sadržinski pregled svih relevantnih odlomaka.

3.3 Tri varijante strukturnih segmenata

Strukturni eksperiment ispituje konfiguracije 256/32, 512/64 i 768/96. Prvi broj označava ciljnu dužinu segmenta, a drugi budžet preklapanja, oba u tokenima kodiranja *cl100k_base*. Na primer, 256/32 znači cilj od 256 tokena i do 32 tokena za prenos celih završnih blokova. Relativni budžet preklapanja u sve tri varijante iznosi jednu osminu ciljne veličine. Veličina i preklapanje menjaju se zajedno, pa eksperiment ne može nezavisno da odredi doprinos svakog parametra.

Blokovi iste sekcije redom se dodaju u tekući segment, a ako sledeći blok menja sekciju ili bi zbir prešao ciljnu veličinu, postojeći segment se upisuje. Unutar iste sekcije moguće je preneti cele završne blokove prethodnog segmenta, dok njihov zbir ostaje u budžetu preklapanja. Zbog toga

stvarno preklapanje nije uvek jednako zadatoj vrednosti i može biti nula, dok se pri promeni sekcije ne prenose blokovi prethodne sekcije.

Preveliki pojedinačni blokovi dele se fiksnim prozorom. Takvi segmenti nose oznaku *contains_partial_block*, kako bi se razlikovali od segmenata sa celim blokovima. U varijanti 256/32 ima 4.643 segmenta sa ovom oznakom, u 512/64 ima 1.406, a u 768/96 ukupno 677. Manja ciljana veličina stoga ne utiče samo na broj segmenata već povećava i broj situacija u kojima se prekida blok.

Dodatna provera sačuvanih artefakata pokazala je da ciljna veličina nije stroga gornja granica. U varijanti 256/32 postoji 144 segmenta iznad 256 tokena, u 512/64 njih 55, a u 768/96 njih 22. Maksimalne izmerene vrednosti su 289, 557 i 856 tokena. Razlog je to što posle vraćanja završnih blokova preklapanja nije ponovo proverena ukupna dužina sa novim blokom i separatorima. Rezultati se zato tumače kao poređenje konkretnih konfiguracija, uz ovo ograničenje implementacije.

Tabela 3.3.1 Stvarne osobine strukturnih segmenata

Konfiguracija	Segmenata	Iznad cilja	Maksimum	Delimičan blok
256/32	36165	144	289	4643
512/64	25103	55	557	1406
768/96	22371	22	856	677

Za *embedding* se ispred teksta segmenta dodaju naslov dokumenta i putanja sekcije. Osnovni tekst segmenta ostaje zasebno polje. Ovo povećava kontekst za vektorsko predstavljanje, posebno za kratke odlomke čije značenje zavisi od naslova. Istovremeno je to još jedan razlog zbog koga naziv konfiguracije ne treba tumačiti kao ukupan broj tokena obrađenih u *embedding* modelu.

Tabela 3.3.2 dopunjuje ekstreme raspodelom stvarnih dužina. Medijana ne raste monotono sa ciljem: mnogi segmenti završavaju se na granici odeljka ili bloka pre popunjavanja budžeta. Poređenje medijane i proseka zato bolje opisuje kolekciju od samog naziva konfiguracije. Vrednosti su ponovo izračunate iz sačuvanog polja *token_count*, bez promene segmenata.

Tabela 3.3.2 Raspodela dužina segmenata u tokenima

Postavka	Prosek	*p25*	Medijana	*p75*	*p95*
256/32	154,73	80	166	234	256
512/64	221,86	75	171	374	509
768/96	247,62	70	160	359	751

3.4 Referentni indeks i poreklo artefakata

Referentni skup sadrži 13.940 segmenata dobijenih u ranijoj fazi projekta. U njemu je korišćena normalizacija *reStructuredText* sadržaja iz istorijskog programskog toka i konfiguracija prozora 512/64. Relevantni istorijski moduli sačuvani su radi praćenja porekla artefakata. Taj skup u daljem tekstu nosi naziv referentni ili *baseline* indeks.

Aktivni *build.py* ima jednostavniju logiku izgradnje iz praznog direktorijuma podataka. On direktno deli izvorni tekst prema tokenima i koristi prazne putanje sekcija. Ako datoteka sa segmentima postoji, program preskače njenu izgradnju. Stoga komanda *setup* nad postojećim paketom koristi sačuvane artefakte, dok izgradnja od početka ne garantuje istovetnu reprodukciju istorijskog referentnog skupa.

Ova razlika je važna za eksperimentalni zaključak. Poređenje referentnog indeksa i Docutils varijanti istovremeno obuhvata promene parsiranja, strukture, metapodataka i teksta za *embedding*. Ono daje poređenje dva postupka pripreme korpusa, dok se kontrolisanije poređenje konfiguracija segmentacije ostvaruje unutar tri Docutils varijante, koje dele istu obradu i modele.

Razlikuju se i leksička polja. Referentni FTS5 indeks pretražuje *text*, *title*, *document_id* i *section_paths* sa težinama 1,0 / 2,0 / 1,5 / 1,2. Strukturni indeks koristi *text*, *title* i *section_path* sa težinama 1,0 / 2,0 / 1,2, dok *source_row* služi samo za povezivanje sa segmentom. Pojavljivanje imena modula u putanji dokumenta stoga može doprineti referentnom BM25 skoru na način koji strukturni indeks nema.

Referentni rezultati u radu predstavljaju analizu arhiviranog stanja. Za njihovo proveravanje čuvaju se datoteke segmenata, mapa redova, indeksi i manifest kontrolnih zbirova. Prisustvo istorijskih modula samo po sebi ne potvrđuje identičnu obnovu svih artefakata. Glavno poređenje konfiguracija zato koristi tri strukturne varijante, uz jasno označen postupak ponovnog čitanja postojećih indeksa.

3.5 Evaluacioni skup

Evaluacioni skup je zapisan u JSONL datoteci. Svaki red sadrži identifikator pitanja, tekst, oznaku odgovorivosti, kategoriju, listu relevantnih dokumenata i referentni odgovor kada je pitanje odgovorivo. Šesnaest pozitivnih pitanja ima po jedan relevantan dokument, dok četiri negativna imaju praznu listu relevantnih dokumenata i nemaju referentni odgovor.

Pitanja pokrivaju preporuku uvlačenja iz PEP 8, *list comprehensions*, *else* granu petlje, nepromenljive dataclass objekte, čitanje teksta pomoću *pathlib*, *Decimal*, *asyncio TaskGroup*, virtuelna okruženja, *lru_cache*, *zoneinfo*, *argparse*, *sqlite3*, *pickle*, *sys.path* i PEP predloge 703 i 695. To daje više tematskih kategorija, ali se iz jednog ili nekoliko pitanja po kategoriji ne može izvesti pouzdana ocena zasebnog domena.

Negativni primeri obuhvataju glavni grad Australije, pritisak u pneumaticima automobila, PostgreSQL VACUUM i temperaturu ključanja vode na Everestu. PostgreSQL primer je bliži domenu softverske dokumentacije od ostalih, ali ceo negativni skup ostaje vrlo mali. Nisu sistematski uključena pitanja o nepostojećim Python funkcijama, pogrešnim pretpostavkama ili ponašanju zavisnom od verzije.

Referentni odgovori služe sadržinskoj proveri pitanja, dok automatska evaluacija pretrage koristi oznake relevantnih dokumenata i odgovorivosti, bez poređenja generisanog teksta sa tim odgovorima. Potpuni pregled pitanja dat je u prilogu A.

Dostupni zapis pitanja ne sadrži dnevnik njihovog nastanka, vreme zaključavanja u odnosu na izbor konfiguracija niti evidenciju nezavisnog pregleda oznaka. Zato se ne pretpostavlja da su uzorkovana iz stvarnih korisničkih zahteva ili nezavisno potvrđena. Pošto se po istom skupu porede i biraju konfiguracije, u ovom radu on ima ulogu razvojnog skupa. Završna procena izabrane konfiguracije zahtevala bi drugi, prethodno zaključan skup.

Dokumentna relevantnost ima dve suprotne slabosti. Segment iz označenog dokumenta može biti nedovoljan za odgovor, dok valjan segment iz drugog dokumenta ostaje neoznačen. Prvi slučaj precenjuje uspeh, a drugi ga potcenjuje. Njihovo razdvajanje zahteva pregled kandidata svih metoda i označavanje raspona koji neposredno potkrepljuju odgovor.

3.6 Modeli i softversko okruženje

Za vektorsko predstavljanje koristi se Jina Embeddings v2 small-en. Model se izvršava na procesoru kroz ONNX Runtime, a izlaz se zapisuje u formatu *float32* i L2 normalizuje. Odgovore generiše Qwen2.5-0.5B-Instruct preko biblioteka Transformers i PyTorch. U oba slučaja učitava se fiksirana lokalna kopija modela. Veze ka korišćenim stanjima modela navedene su u prilogu B radi ponavljanja eksperimenta.

Lokalna konfiguracija Jina modela navodi četiri skrivena sloja, osam glava pažnje, širinu skrivenog stanja 512 i unutrašnju dimenziju *feed-forward* dela 2.048. Reprezentacija teksta dobija se prosečnim objedinjavanjem tokena, što potvrđuje i konfiguracija *pooling* komponente. U aplikaciji se koristi već izvezen ONNX model sa ovim objedinjavanjem, a zatim se izlaz L2 normalizuje.

Konfiguracija generatora navodi arhitekturu *Qwen2ForCausalLM* sa 24 skrivena sloja, širinom 896, 14 glava pažnje i dve glave za ključeve i vrednosti. Unutrašnja dimenzija iznosi 4.864, a deklarisana granica pozicija 32.768. To su svojstva učitane mreže, a ne postavke eksperimenta: aplikacija koristi samo dva kontekstna segmenta i ograničava generisanje na 192 nova tokena. Nijedna od ove dve mreže nije dodatno obučavana u okviru projekta.

Tabela 3.6.1 Zabeležene verzije softverskog okruženja

Komponenta	**Verzija**
Python	3.14.3
macOS / arhitektura	26.6.2 / ARM64
Docutils	0.23
FAISS	1.15.0
NumPy	2.5.3
ONNX Runtime	1.24.4
tiktoken	0.14.0
PyTorch	2.14.0
Transformers	4.57.6

Inventar priložen eksperimentu navodi Python 3.14.3 na macOS 26.6.2, ARM64, i verzije glavnih biblioteka prikazane u tabeli. To je zapis okruženja uz sačuvane rezultate. U istom

inventaru nisu zabeleženi tačan model procesora i količina radne memorije, pa se oni ne pretpostavljaju na osnovu računara na kome se rad naknadno priprema. Odsustvo tih podataka ograničava poređenje latencije sa drugim platformama.

Prosečno objedinjavanje *encoder* izlaza koristi masku: predstave dopunskih tokena isključuju se iz zbira i broja tokena. Tako vektor kratkog teksta ne zavisi neposredno od broja mesta dodatih radi izjednačavanja dužina u grupi. Qwen radi drugačije: na osnovu prethodnih tokena pitanja, konteksta i započetog odgovora računa raspodelu sledećeg tokena. U determinističkom režimu bira se token najvećeg skora, sve do završne oznake ili postavljene granice.

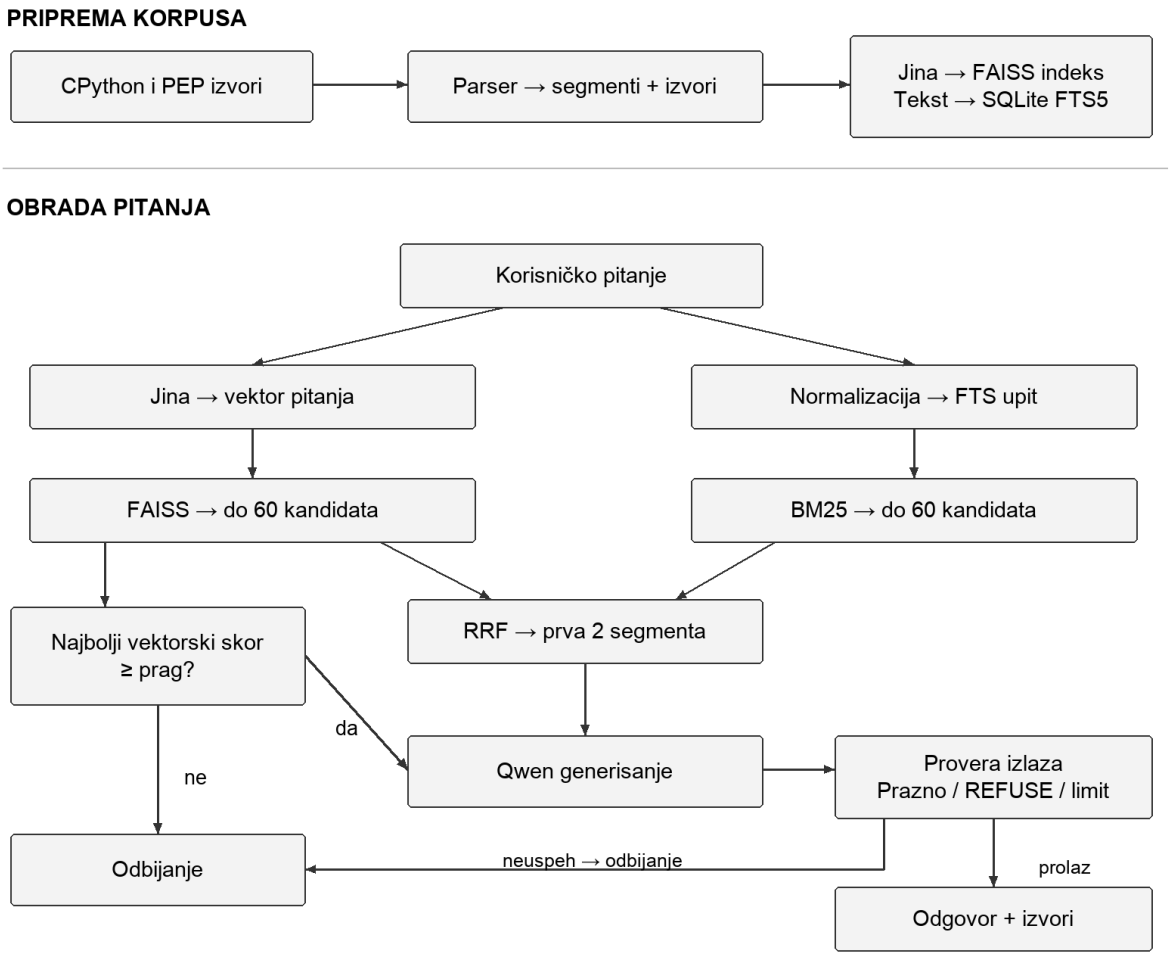
Mala varijanta generatora omogućava lokalni demonstracioni tok, ali broj parametara sam ne određuje kvalitet odgovora. Na ovom projektu izbor modela nije potvrđen uporednim eksperimentom sa većim generatorom. Dokumentovani kriterijumi primene su engleski korpus, dostupna lokalna revizija i podrška korišćenog izvršnog okruženja, dok se detaljniji istorijski razlog izbora ne može rekonstruisati samo iz konfiguracionih datoteka.

4 REALIZACIJA SISTEMA

Realizacija sistema opisana je od arhitekture i toka podataka do generisanja odgovora. Najpre se prikazuje raspodela poslova između modula za pripremu, pretragu i pokretanje komandi, a zatim se izdvajaju implementacioni problemi koji su tražili posebna rešenja: pristup segmentima preko indeksa, obrada nepoznatih direktiva u izvoru, kontrola dužine segmenata, spajanje rang-lista nezavisno od skale skorova i upravljanje memorijom pri izvršavanju modela. Opis je napisan na osnovu izvornog koda aplikacije i dokumentacije korišćenih biblioteka.

4.1 Arhitektura i tok podataka

Slika 4.1.1 prikazuje tok lokalne Python aplikacije sa komandnim interfejsom. Modul *build.py* priprema podatke i referentne indekse, *rag.py* realizuje pretragu, generisanje i pokretanje komandi, a *evaluation/chunk_sizes.py* gradi i ispituje strukturne varijante. Odvojeni modul proverava integritet artefakata. Ova podela omogućava da se skupa izgradnja indeksa obavi unapred i da se pitanja postavljaju nad već pripremljenim podacima.



Slika 4.1.1 Tok obrade korpusa i odgovaranja na pitanje

U fazi izgradnje učitavaju se izvorni dokumenti i formiraju segmenti. Njihov tekst prevodi se u vektore koji ulaze u FAISS, dok tekstualna polja ulaze u SQLite FTS5. Prilikom upita pitanje se takođe prevodi u vektor. Obe metode vraćaju do 60 kandidata, nakon čega se liste spajaju pomoću RRF. Za rad aplikacije zadržava se pet rezultata, a generator dobija prva dva.

Pre generisanja primenjuje se prag nad najboljim vektorskim skorom. Ako uslov nije ispunjen, aplikacija vraća poruku da ne može odgovoriti iz ponuđenog konteksta. Ako jeste, dva segmenta postaju izvori S1 i S2 u upitu generatoru. Konačan izlaz sadrži *status*, tekst odgovora, izvore i, u JSON režimu, sadržaj segmenata korišćenih za generisanje.

Eksperimentalne Docutils varijante nisu povezane sa izborom indeksa u komandi *ask*. Putanje u *rag.py* pokazuju na referentni indeks. Zato pronalazak najbolje konfiguracije u eksperimentu ne znači da je ona automatski postala aktivna u korisničkom toku. Za takvo

uključivanje potreban je izbor konfiguracije i prilagođavanje učitavanja različitih formata metapodataka.

4.2 Identitet i pristup segmentima

Referentni vektorski indeks prati datoteka *rows.jsonl*. Ona mapira red vektora na identifikator segmenta, kontrolni zbir njegovog teksta i bajtski pomeraj u datoteci *chunks.jsonl*. Funkcija *chunk* otvara datoteku u binarnom režimu, pozicionira se na taj pomeraj i čita jedan red. Binarni režim je bitan zato što se pomeraj izražava u bajtovima, dok broj znakova UTF-8 teksta ne mora odgovarati broju bajtova.

Ispis 4.2.1 Pristup segmentu preko bajtskog pomeraja (*rag.py*, red 75)

```
def chunk(row: dict) -> dict:
    with CHUNKS.open("rb") as stream:
        stream.seek(row["chunks_jsonl_byte_offset"])
        return json.loads(stream.readline())
```

Ovako se ne mora učitavati celokupan tekst svih referentnih segmenata da bi se prikazalo nekoliko rezultata. Mapa redova se kešira u procesu. Tačnost veze između vektora i teksta zavisi od nepromenjenog redosleda i sadržaja datoteka, zbog čega je uvedena posebna provera identifikatora i SHA-256 vrednosti. Provera strukture indeksa bez provere teksta ne bi otkrila sve moguće pogrešne veze.

Strukturni eksperiment učitava listu segmenata i pristupa joj pozicijama koje vrati FAISS. Segmenti imaju identifikatore izvedene iz konfiguracije, dokumenta, redosleda i teksta. Ako obrada promeni tekst ili redosled segmenata, menjaju se i njihovi identifikatori. Takvi identifikatori služe povezivanju podataka unutar konfiguracije i ne treba ih tumačiti kao trajne oznake istog odlomka kroz sve buduće verzije parsera.

4.3 Obrada nepoznatih direktiva

Jedan od konkretnih problema pri pripremi korpusa jeste prisustvo direktiva koje osnovni Docutils parser ne prepoznaje. Odbacivanje takvih delova moglo bi ukloniti objašnjenja funkcija ili primere. Realizovano rešenje registruje opštu direktivu *PreserveDirective* i njome čuva argument i unutrašnji sadržaj.

Ispis 4.3.1 Očuvanje sadržaja nepoznate direktive (*evaluation/chunk_sizes.py*, red 52)

```
def run(self):
    container = nodes.container()
    if self.arguments:
        container += nodes.paragraph(text=self.arguments[0])
    self.state.nested_parse(self.content, self.content_offset, container)
    return [container]
```

Direktiva smešta argument i parsirani sadržaj u isti kontejner i tako čuva tekst koji je važan pretrazi. Ono ne reprodukuje specifično ponašanje svake direktive, kao što je razrešavanje svih unakrsnih veza. Ta razlika mora se uzeti u obzir kada se objašnjava zašto strukturno parsiranje nije isto što i potpuno renderovanje dokumentacije.

4.4 Ograničenje kontrole dužine segmenata

Drugi problem je usklađivanje granica blokova sa ciljnom dužinom segmenta. Kod proverava da li kandidat sa novim blokom prelazi ciljnu veličinu ili menja sekciju. Ako se segment završi unutar iste sekcije, pokušava da vrati njegove završne blokove u novi segment, ali samo do budžeta preklapanja.

Ispis 4.4.1 Prenos završnih blokova u naredni segment (*evaluation/chunk_sizes.py*, red 164)

```
if current and (changed_section or len(encoding.encode_ordinary(candidate)) > size):
    old = current
    emit(document, current_path, old)
    current = []
    if not changed_section:
        used = 0
        for part in reversed(old):
            count = len(encoding.encode_ordinary(part))
            if used + count > overlap:
                break
            current.insert(0, part)
            used += count
if not current:
    current_path = path
current.append(block["text"])
```

U poslednjem koraku novi blok se dodaje bez ponovne provere ukupne dužine, pa nastaju prekoračenja evidentirana u trećem poglavlju. Prekoračenja su zabeležena u polju *token_count*, koje koristi isti tokenizator kao segmentacija. U tabelama su rezultati verzije koja dopušta ta prekoračenja.

Moguće poboljšanje je da se dužina proveri i posle izbora preklapanja, pa da se preneti blokovi uklanjaju dok segment ne zadovolji ograničenje dužine. Takva izmena nije primenjena na artefakte analizirane u radu. Njena evaluacija zahtevala bi ponovnu izgradnju sva tri indeksa i ponavljanje merenja, kako se rezultati stare i nove obrade ne bi mešali.

4.5 Spajanje nezavisno od skale skorova

Treći problem je objedinjavanje rezultata različitih pretraživača. Funkcija *rrf* vodi rečnik po *chunk_id*, pamti pozicije iz obe metode i dodaje recipročni doprinos ranga. Konačno sortiranje koristi zbir, a kod jednakog zbira vektorski pa leksički rang kao deterministično razrešenje.

Ispis 4.5.1 Spajanje rezultata prema rangovima (*rag.py*, red 59)

```
def rrf(dense: list[dict], sparse: list[dict], k: int = 60) -> list[dict]:
    merged = {}
    for name, values in (("dense", dense), ("bm25", sparse)):
        for rank, row in enumerate(values, 1):
            item = merged.setdefault(
                row["chunk_id"],
                {**row, "dense_rank": None,
                 "bm25_rank": None, "score": 0.0},
            )
            item[f"{name}_rank"] = rank
            item["score"] += 1 / (k + rank)
    return sorted(
        merged.values(),
        key=lambda row: (
            -row["score"],
            row["dense_rank"] or 10**9,
            row["bm25_rank"] or 10**9,
        ),
    )
```

Ovaj postupak je neposredno proverljiv malim primerima. Test koji menja samo skale ulaznih skorova potvrđuje da se izlazni redosled ne menja, dok test saglasnosti potvrđuje da zajednički segment može da bude prvi. Testovi ne dokazuju da će fuzija poboljšati relevantnost. To nije pitanje za jedinične testove, nego za eksperimentalnu evaluaciju.

4.6 Izvršavanje modela i upravljanje memorijom

Pri računjanju *embedding* vektora tekstovi se najpre sortiraju po dužini. Veličina grupe zavisi od dužine najdužeg teksta, uz zadatu gornju granicu broja elemenata. Na taj način se smanjuje dopunjavanje kratkih sekvenci do mnogo duže sekvence u istoj grupi i ograničava veličina ulaznih tenzora. Vektori se zatim vraćaju na originalne pozicije, kako bi se očuvala veza sa segmentima.

Referentni tok koristi odvojene procese za pojedine faze pretrage i FAISS operaciju. Vektor pitanja privremeno se zapisuje kao NumPy datoteka, a podproces vraća pozicije i skorove u JSON formatu. Ta organizacija jasno razdvaja izvršavanje, ali donosi trošak pokretanja procesa i ponovnog učitavanja resursa. Zato se latencija korisničke komande ne može poistovetiti sa merenjem funkcije za pretragu u eksperimentu.

U eksperimentalnom programu indeks je već učitan kada počne merenje pretrage. Pitanja su prethodno prevedena u vektore. Razlika između dva načina izvršavanja ne utiče na definiciju rangiranja, ali neposredno utiče na to šta obuhvata objavljeno vreme. Praktično ubrzanje aplikacije zahtevalo bi merenje celog toka i razmatranje trajno učitanih modela i indeksa.

Strukturni tok ograničava grupu na najviše 64 teksta. Za najdužu sekvencu sa L tokena broj elemenata dodatno ograničava izraz max(1, ⌊20.000.000/L²⌋), uz broj preostalih tekstova. To je heuristika iz implementacije, a ne merenje zauzeća memorije. Razlog istorijskog uvođenja podprocesa nije zabeležen, pa se razdvajanje procesa ovde opisuje kao arhitekturna osobina sa troškom pokretanja, bez pripisivanja nedokumentovanih kvarova.

4.7 Pravilo odbijanja i generisanje

Prag za pokušaj odgovora iznosi 0,837121456861496 i primenjuje se na najbolji vektorski skor. Njegovo poreklo je arhivirana kalibracija sa deset pozitivnih i deset negativnih pitanja. U tom skupu izabrani prag razdvaja svih dvadeset primera. Kalibracioni i kasniji evaluacioni upiti nisu ista lista, ali sadrže bliske teme, na primer PEP 8 i virtuelna okruženja. Stoga se ne može pretpostaviti potpuna nezavisnost razvoja i procene.

Generator dobija instrukciju da odgovara iz izvora, zanemari instrukcije koje se eventualno nalaze u njima i vrati *REFUSE* kada izvori nisu dovoljni. Traži se odgovor do 80 reči na jeziku pitanja. Programska granica je 192 nova tokena, a broj reči se posebno ne proverava. Isključeno je uzorkovanje, pa se koristi deterministički izbor sledećeg tokena u zadatoj konfiguraciji.

Pored odbijanja pre generisanja, postoje dodatni uslovi nakon njega. Prazan odgovor, odgovor jednak *REFUSE* ili dostizanje ograničenja od 192 generisana tokena pretvaraju izlaz u odbijanje. Dostizanje ograničenja tretira se konzervativno, jer tekst može biti nedovršen. Takvo odbijanje ne mora značiti da odgovor nije postojao u korpusu.

Iz generisanog teksta uklanjaju se oznake oblika S1 i S2, a u izlaz se programski dodaju oba izabrana izvora. To obezbeđuje pregled konteksta, ali ne uspostavlja vezu svake tvrdnje sa konkretnim dokazom. Izvor može biti naveden i kada ga model stvarno nije koristio. Zato prisustvo citata nije rezultat evaluacije njihove ispravnosti.

Prag proverava vektorsku listu, dok se kontekst bira iz RRF liste. Najbolji vektorski kandidat zato ne mora biti prosleđen modelu. Prihvatanje znači da postoji dovoljno visok vektorski skor prema zadatoj heuristici, ali ne potvrđuje da oba izabrana segmenta potkrepljuju odgovor.

Postojeća provera jednakosti sa *REFUSE* ne obuhvata automatski varijantu „*REFUSE*.“ sa tačkom. Takođe, *status refused* objedinjuje nizak skor, prazan izlaz, eksplicitno odbijanje modela i dostizanje granice tokena. Za zasebnu statistiku ovih uzroka bilo bi potrebno interno polje razloga odbijanja i testovi graničnih izlaza. U radu se uspeh praga zato odvaja od ponašanja generatora.

5 METODOLOGIJA ANALIZE I TESTIRANJA

Ocenjivanje pretrage počiva na unapred određenom eksperimentalnom planu, merama i postupcima provere. Redom se opisuje kako se određuju relevantnost i položaj rezultata, koje se metrike uspešnosti koriste, kako se mere latencija i veličina indeksa i na koji se način procenjuju pravilo odbijanja i funkcionalna ispravnost. Izbor metrika i način ocenjivanja prate ustaljenu praksu u vrednovanju pronalaženja informacija [9, 12].

5.1 Eksperimentalni plan

Osnovno poređenje obuhvata tri konfiguracije strukturne segmentacije i tri metode pretrage. Za svaku konfiguraciju grade se odgovarajući FAISS i SQLite indeksi. Nad istih 16 odgovorivih pitanja računaju se rezultati vektorske pretrage, BM25 i RRF fuzije. Tako nastaje devet kombinacija, bez promene *embedding* modela ili evaluacionih oznaka između njih.

Isti izvorni korpus, parser, način formiranja *embedding* teksta, dimenzija vektora, broj kandidata i konstanta fuzije koriste se u sve tri strukturne varijante. Menjaju se ciljna veličina segmenta i budžet preklapanja, dok broj segmenata, njihova stvarna dužina i sadržaj predstavljaju posledice te promene. Nije izvršena nezavisna ablacija naslova, preklapanja ili parsera.

Referentni indeks evaluira se zasebno, nad istim pitanjima. On služi za opis polaznog stanja i poređenje kvaliteta različitih postupaka pripreme. Zbog razlika u njegovom poreklu, vrednosti se ne koriste kao dokaz izolovanog efekta veličine segmenta. Nije izvedeno poređenje generativnog modela bez konteksta i sa kontekstom.

Pregled postavki u tabeli 5.1.1 odvaja istorijski referentni skup od glavnog strukturnog poređenja. Kriterijum izbora konfiguracije u ovom radu je najveći hibridni *MRR* na razvojnom skupu. Prostor i latencija prikazuju cenu tog izbora, bez naknadnog definisanja zajedničkog skora kvaliteta i troška.

Tabela 5.1.1 Pregled eksperimentalnih postavki

Osobina	Istorijski reper	Strukturno poređenje
Segmentacija	512/64, ranija normalizacija	Docutils: 256/32, 512/64, 768/96
Leksičke težine	*text* 1; *title* 2; *document_id* 1,5; *section_paths* 1,2	*text* 1; *title* 2; *section_path* 1,2
Vektori i indeks	Jina small-en, 512, L2; *IndexFlatIP*	Isti model i tip indeksa
Kandidati i fuzija	60 + 60; RRF c = 60	60 + 60; RRF c = 60
Pitanja za pretragu	16 odgovorivih	Istih 16 odgovorivih
Prag i generator	Prag na 20 pitanja; *ask* koristi ovaj indeks	Prag i generator van eksperimenta

5.2 Relevantnost i položaj rezultata

Za svako odgovorivo pitanje unapred je označen relevantan dokument. Rezultati pretrage su segmenti, a funkcija za evaluaciju pronalazi prvu poziciju na kojoj *document_id* segmenta pripada skupu relevantnih dokumenata. Ta pozicija je rang prvog relevantnog rezultata. Ako nijedan kandidat nije iz označenog dokumenta, doprinos recipročnom rangu je nula.

Važno je da se pre računanja ne uklanjaju ponovljeni dokumenti. Ako prvih pet segmenata potiče iz istog nerelevantnog dokumenta, oni zauzimaju svih pet pozicija. Obrnuto, segment koji potiče iz relevantnog dokumenta računa se kao pogodak čak i ako njegov tekst ne sadrži tačan odgovor. Obeležavanje na nivou dokumenta zato može preceniti kvalitet dokaznog konteksta.

5.3 Metrike uspešnosti pretrage

Kroz rad se koristi naziv *Success@k*: udeo pitanja za koja se među prvih k segmenata nalazi segment iz označenog dokumenta. Izvorne datoteke ovu meru nazivaju *recall@k*. Pošto svako pitanje ima jedan označen dokument, mera odgovara pogotku tog dokumenta u skupu dokumenata zastupljenih među prvih k segmenata.

$$Success@k = \frac{1}{N}\sum_{i=1}^{N} I(r_i \leq k) \tag{5.3.1}$$

U jednačini 5.3.1 N je broj odgovorivih pitanja, a r_i rang prvog segmenta iz označenog dokumenta. Ako dokument nije pronađen, definiše se $r_i = \infty$. Indikator I ima vrednost jedan kada je uslov ispunjen, odnosno nula inače. Za N = 16 jedno pitanje menja *Success@k* za 0,0625.

MRR daje veći doprinos ranijem pogotku. U jednačini 5.3.2 koristi se recipročni rang: prvi rezultat doprinosi sa jedan, drugi sa 0,5, a peti sa 0,2.

$$MRR = \frac{1}{N}\sum_{i=1}^{N} RR_i \quad (5.3.2)$$

Ovde je $RR_i = 1/r_i$ kada je relevantan dokument pronađen u listi kandidata, a $RR_i = 0$ kada nije pronađen.

Vektorska i leksička lista imaju do 60 kandidata, a fuzija do 120 različitih segmenata. Radi zajedničkog poređenja dodatno se računa *MRR@60*: doprinos je nula ako je prvi pogodak iza ranga 60. Za referentne rangove ovo ograničenje ne menja objavljeni *MRR*, jer su svi pogoci u prvih 60. Za strukturne varijante proverava se na ponovljenim detaljnim rangovima.

Success@2 odgovara broju segmenata koje aplikacija prosleđuje generatoru. Računa se uz *Success@1* i *Success@5*, tako da izbor mere pokazuje razliku između ranog pogotka i šire liste rezultata. Ni pri k = 2 dokumentna oznaka ne potvrđuje sadržaj samog segmenta, a ta provera pripada zasebnom ocenjivanju konteksta.

5.4 Merenje latencije i veličine indeksa

Latencija se meri pomoću *time.perf_counter*. Vektorski interval obuhvata poziv pretrage nad učitanim FAISS indeksom, a leksički SQL pretragu i preuzimanje rezultata. Hibridni interval počinje pre vektorske pretrage i završava se nakon RRF spajanja, pa uključuje i međukorake pripreme rezultata u Pythonu.

Vektori svih pitanja izračunati su pre početka merenja, a indeksi su već učitani. Mereni interval obuhvata pretragu, dok se priprema pitanja, učitavanje modela, generisanje odgovora i pokretanje komandnog procesa nalaze izvan tog intervala.

Za svaku metodu i konfiguraciju postoje vrednosti za 16 pitanja. Prikazani su *p50*, odnosno medijana, i *p95*, procena granice ispod koje je približno 95% trajanja u uzorku. Izračunati su NumPy percentile funkcijom, uz interpolaciju između susednih opažanja. Nema zasebne faze zagrevanja ni više ponovljenih serija. *P95* je zbog malog broja uzoraka naročito osetljiv na pojedinačna sporija izvršavanja.

Veličina vektorskog indeksa meri se kao veličina FAISS datoteke na disku i prikazuje u megabajtima. Za *IndexFlatIP* sa 512 *float32* komponenti osnovni sadržaj vektora zauzima 2.048 bajtova po segmentu, uz dodatne metapodatke formata. Prikazana veličina ne uključuje SQLite, tekst segmenata, modele i memoriju procesa. Ona je uporediv trošak jedne komponente sistema.

5.5 Procena pravila odbijanja

Pravilo odbijanja se procenjuje na svih 20 pitanja referentnog eksperimenta. Poredi se odluka da najbolji vektorski skor prelazi prag sa oznakom *answerable*. Broje se ispravno prihvaćena i ispravno odbijena pitanja, kao i moguća pogrešna odbijanja i prihvatanja.

Ovaj test meri samo odluku pre generisanja. Ako je pitanje prihvaćeno, evaluator ne pokreće generator i ne proverava odgovor. Takođe ne meri kasnije odbijanje izazvano praznim izlazom ili dostignutim ograničenjem tokena. Naziv *accuracy* u rezultatu zato se odnosi na tačnost klasifikacije odgovorivosti datih primera.

5.6 Funkcionalna provera

Funkcionalna provera ima dva nivoa. Prvi proverava artefakte: redosled mapiranja, identifikatore segmenata, SHA-256 tekstova, broj vektora, njihovu dimenziju, integritet SQLite baze i broj njenih redova. Drugi nivo čini šest jediničnih testova za normalizaciju upita, formiranje FTS izraza i RRF logiku.

Tabela 5.6.1 Obuhvat postojećih jediničnih testova

Test	Provereno ponašanje
test_terms	Proširenje PEP8 u pep8, pep i 8
test_fts	Formiranje izraza sa operatorom OR
test_empty_fts	Prazan rezultat za upit bez termina
test_rrf_agreement	Prednost segmenta prisutnog u obe liste
test_rrf_ignores_score_scale	Nezavisnost fuzije od skale skorova
test_rrf_ranks	Čuvanje rangova iz obe metode

Pri proveri paketa verifikacija integriteta završena je izlazom OK i svih šest testova je prošlo, čime su potvrđene proverene osobine postojeće verzije. Testovi ne pokrivaju sve grane parsera, prekoračenje ciljne veličine strukturnih segmenata, tačnost generacije ili kvalitet citata. Uspeh ovih testova zato se navodi odvojeno od eksperimentalnih metrika.

5.7 Ponovljivost i ograničenja eksperimentalnog postupka

Analiza razlikuje ponavljanje pretrage nad arhiviranim indeksima od pune izgradnje. Prvo proverava usklađenost podataka i metrika uz fiksne segmente. Drugo obuhvata parser, tokenizaciju i

modele i zahteva zaseban izlazni direktorijum. Postojeće datoteke ne treba neselektivno brisati: njihov identitet se čuva manifestom, a nova verzija dobija odvojene putanje.

U dopunskoj reviziji pretraga je ponovljena nad trima sačuvanim strukturnim indeksima. Izračunati su vektori istih 16 pitanja lokalnom revizijom modela i sačuvane pune rang-liste svih devet kombinacija. Pre pretrage provereni su broj i dimenzija FAISS vektora i jednakost svih SQLite tekstova, naslova i putanja sa odgovarajućim redovima datoteke segmenata. Ova provera ne dokazuje ponovnim računanjem da je svaki postojeći vektor dobijen baš iz tog teksta, jer bi za to bila potrebna ponovna izgradnja.

Dopunski proračun ima sopstveni datum, skriptu, kontrolne zbirove indeksa i detaljne izlaze. Njime se proveravaju zbirne metrike i dodaju *Success@2* i *MRR@60*, bez zamene istorijske serije latencija. Nisu računati intervali poverenja ni testovi značajnosti: nalazi opisuju ponašanje konfiguracija na malom razvojnom skupu.

5.8 Postupak analize odgovora u studiji slučaja

Studija slučaja obuhvata dva demonstraciona upita i tri dopunska sačuvana izlaza. Dopunska pitanja q02, q12 i q14 odabrana su pre generisanja odgovora prema poznatim rangovima: promašaj označenog dokumenta, razlikovanje dve operacije istog API-ja i pogoršanje ranga nakon fuzije. Takav nameran izbor služi analizi grešaka i ne predstavlja nezavisan testni uzorak.

Za svaki izlaz proveravaju se tačnost osnovne tvrdnje, potpunost odgovora na pitanje i podrška u dva prosleđena segmenta. Posebno se proverava da li navedeni izvori sadrže tu podršku i da li je eventualno odbijanje opravdano. Izvorni JSON izlazi sačuvani su bez prepravljanja. Rezultati su pojedinačne sadržinske analize bez nezavisnog drugog ocenjivača, pa se iz njih ne izvodi procenat tačnih odgovora celog sistema.

6 REZULTATI I DISKUSIJA

Rezultati eksperimenata izlažu se i analiziraju prema istraživačkim pitanjima iz uvoda. Najpre se prikazuju referentne metode pretrage i analiza pojedinačnih pitanja, zatim uticaj konfiguracije strukturnih segmenata na uspešnost, broj segmenata i veličinu indeksa, kao i zabeležena latencija i ponašanje pravila odbijanja. Poslednji odeljak razmatra ograničenja i validnost nalaza, dok izvorne vrednosti svih merenja čuva elektronski prilog.

6.1 Referentne metode pretrage

Referentni indeks sadrži 13.940 segmenata. U tabeli 6.1.1 prikazani su rezultati vektorske, BM25 i hibridne pretrage nad 16 odgovorivih pitanja. Vrednosti su zaokružene na četiri decimale, dok izvorni JSON čuva veću preciznost.

Tabela 6.1.1 Rezultati referentnog indeksa

Metoda	***Success@1***	***Success@2***	***Success@5***	***MRR@60***
Vektorska	0,4375	0,7500	0,9375	0,6493
BM25	0,5000	0,5625	0,6875	0,6024
Hibridna	0,5000	0,6875	0,9375	0,6667

Vektorska pretraga nalazi relevantan dokument na prvom mestu za sedam od 16 pitanja, a BM25 i hibridna za osam. Među prvih pet rezultata vektorska i hibridna metoda uspevaju za 15 pitanja, a BM25 za 11. Hibridna metoda ostvaruje najveći *MRR*, 0,6667, naspram 0,6493 za vektorsku i 0,6024 za BM25.

Razlika hibridne i vektorske metode po *MRR* iznosi približno 0,0174. Ona pokazuje određeno poboljšanje rasporeda rezultata na ovom skupu, uz isti *Success@5*. Sama vrednost ne govori da je svako pitanje poboljšano. Detalji pokazuju da fuzija istovremeno pomera neke relevantne rezultate naviše, a druge naniže.

Za kontekst od dva segmenta poredak je drugačiji: vektorska metoda uspeva za 12/16 pitanja (0,7500), hibridna za 11/16 (0,6875), a BM25 za 9/16 (0,5625). Veći hibridni *MRR* zato ne opravdava zaključak da je kontekst generatora češće relevantan. *MRR* vrednuje pomeranja na celoj listi, dok *Success@2* proverava prelazak konkretne granice.

6.2 Analiza pojedinačnih pitanja

Tabela 6.2.1 daje pojedinačne rangove svih 16 odgovorivih pitanja za tri metode na referentnom indeksu. Poređenje po pitanju pokazuje gde fuzija pomaže, a gde odmaže, što prosečne metrike same ne otkrivaju.

Tabela 6.2.1 Svi rangovi u referentnom eksperimentu

Pitanje	Vektorska	BM25	Hibridna
q01	4	1	1
q02	18	9	20
q03	1	1	1
q04	2	1	1
q05	2	2	1
q06	2	6	2
q07	1	4	2
q08	2	16	4
q09	1	1	1
q10	3	1	3
q11	1	1	1
q12	1	1	1
q13	1	1	1
q14	1	14	5
q15	4	3	2
q16	2	7	3

Za pitanje q01 o broju razmaka prema PEP 8, BM25 nalazi relevantan dokument na prvom mestu, a vektorska metoda na četvrtom. Fuzija ga dovodi na prvo mesto. Sličan dobitak vidi se kod q04, gde vektorski rang dva postaje hibridni rang jedan, i kod q05, gde obe pojedinačne metode imaju prvi relevantan dokument na drugom mestu, a fuzija na prvom.

Pitanje q02 o *list comprehension* konstrukciji promašuje označeni dokument u prvih pet rezultata kod svih metoda. Dokument je na rangu 18 za vektorsku, devet za BM25 i 20 za hibridnu metodu. Pregled stvarnog konteksta pokazuje da raniji segmenti ipak sadrže objašnjenje u

istorijskom dokumentu „What's New in Python 2.0“, koji nije obuhvaćen oznakom relevantnosti. Detalji u odeljku 7.5 razdvajaju promašaj postojećeg evaluacionog ključa od nedostatka korisnog teksta.

Za q14 o prvoj stavci *sys.path* vektorska metoda daje rang jedan, BM25 rang 14, a fuzija rang pet. Označeni dokument time ispada iz konteksta od dva segmenta. Ipak, prosleđeni PEP i dokumentacija komandne linije sadrže traženi podatak, što potvrđuje analiza u odeljku 7.7. Pogoršanje dokumentnog ranga u ovom primeru ne dovodi do pogrešnog odgovora, pa su za ocenu konteksta potrebne potpunije oznake.

Pitanje q10 ima BM25 rang jedan, vektorski tri i hibridni tri. Time se vidi da ni odličan prvi leksički rezultat ne mora ostati prvi nakon fuzije. Rezultati q03, q09, q11, q12 i q13 imaju rang jedan kod sve tri metode i predstavljaju slučajeve njihove saglasnosti.

Puni rangovi u tabeli 6.2.1 pokazuju da RRF u odnosu na vektorsku pretragu poboljšava četiri pitanja, ostavlja sedam na istom rangu i pogoršava pet. To je saglasno rastu *MRR*: pomeranje sa četvrtog na prvo mesto donosi veću promenu recipročnog doprinosa nego pomeranje sa osamnaestog na dvadeseto. Broj poboljšanih pitanja zato treba čitati uz veličinu pomeranja, a ne umesto nje.

6.3 Uticaj konfiguracije strukturnih segmenata

Tabela 6.3.1 prikazuje sve kombinacije metode i konfiguracije, uključujući dopunski izračunat *Success@2*. Vrednosti se odnose na istih 16 odgovorivih pitanja i omogućavaju poređenje tri strukturne konfiguracije po istim merama. Na osnovu njih razmatra se kako ciljna veličina segmenta utiče na rangiranje relevantnih dokumenata.

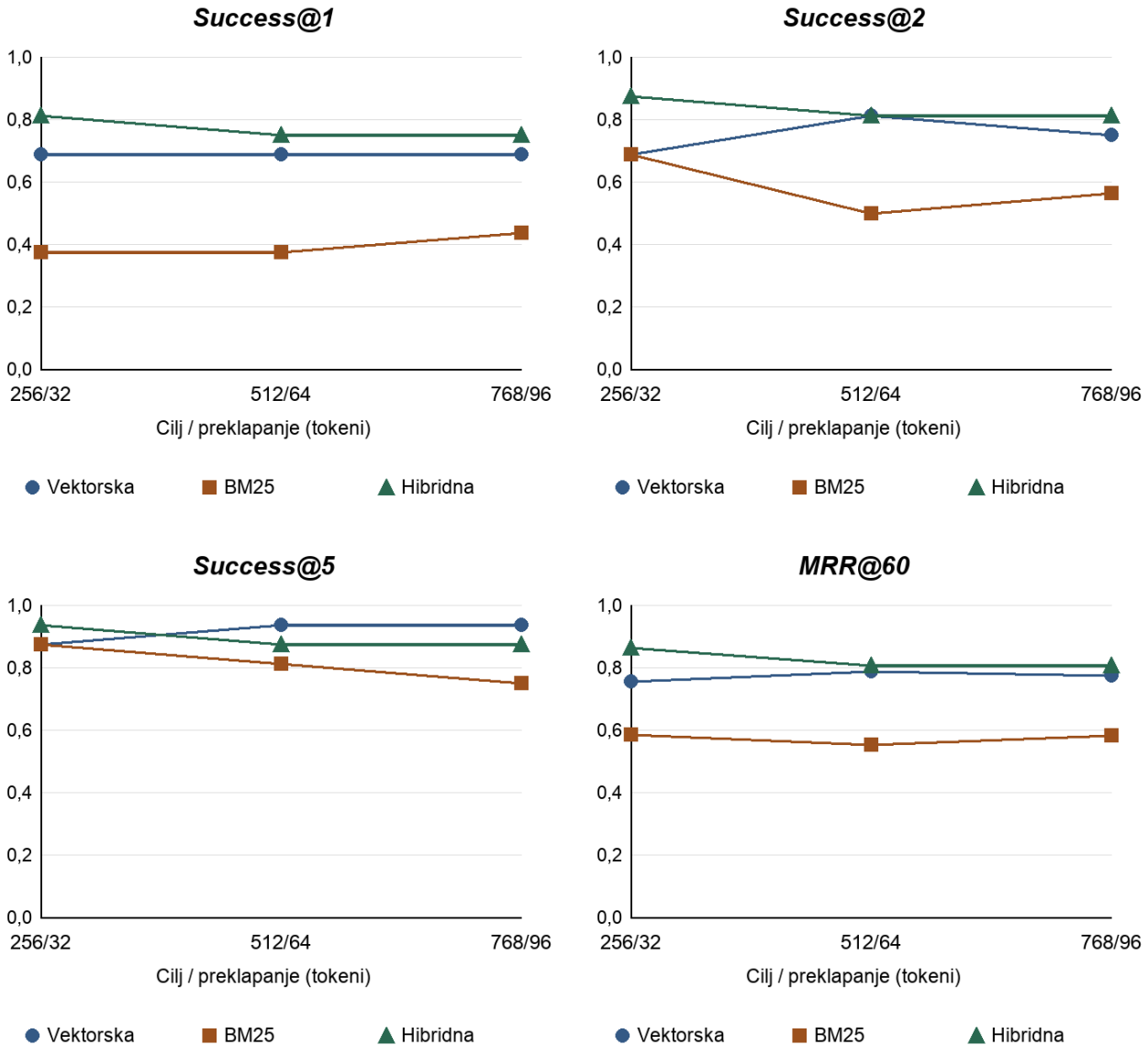
Tabela 6.3.1 Rezultati strukturnih konfiguracija

Postavka	**Metoda**	***Success@1***	***Success@2***	***Success@5***	***MRR@60***
256/32	Vektorska	0,6875	0,6875	0,8750	0,7563
256/32	BM25	0,3750	0,6875	0,8750	0,5851
256/32	Hibridna	0,8125	0,8750	0,9375	0,8642
512/64	Vektorska	0,6875	0,8125	0,9375	0,7891
512/64	BM25	0,3750	0,5000	0,8125	0,5523
512/64	Hibridna	0,7500	0,8125	0,8750	0,8073
768/96	Vektorska	0,6875	0,7500	0,9375	0,7735
768/96	BM25	0,4375	0,5625	0,7500	0,5831
768/96	Hibridna	0,7500	0,8125	0,8750	0,8057

Među hibridnim konfiguracijama najveći *Success@1* i *MRR* ostvaruje 256/32. Relevantan dokument nalazi se na prvom mestu za 13 od 16 pitanja, dok konfiguracije 512/64 i 768/96 uspevaju za 12. Razlika *Success@1* između prve i preostale dve konfiguracije zato odgovara jednom pitanju, odnosno 6,25 procentnih poena.

Na prvih pet rezultata hibridna konfiguracija 256/32 uspeva za 15 pitanja, a preostale dve za 14. Njena *MRR* vrednost 0,8642 veća je od 0,8073 i 0,8057. Konfiguracije 512/64 i 768/96 imaju iste vrednosti *Success@1* i *Success@5*, a mala razlika *MRR* znači da se razlikuju u tačnim pozicijama relevantnih rezultata.

Na slici 6.3.1 upoređene su četiri mere za iste konfiguracije. Različiti markeri omogućavaju razlikovanje metoda i pri crno-beloj štampi.



Slika 6.3.1 Efektivnost metoda u strukturnim konfiguracijama

Kod vektorske pretrage najveći *MRR* ima konfiguracija 512/64, sa vrednošću 0,7891. *Success@1* iznosi 0,6875 u sve tri konfiguracije. *Success@5* iznosi 0,9375 za 512/64 i 768/96, a 0,8750 za 256/32. Prednost manje ciljne dužine zato zavisi od metode i metrike.

Hibridna pretraga ima veći *MRR* od obe pojedinačne metode u sve tri konfiguracije. U varijantama 512/64 i 768/96 njen *Success@5* je niži od vektorskog: među prvih pet rezultata nalazi označeni dokument za jedno pitanje manje. Fuzija tako povećava *MRR* uz gubitak na drugoj meri.

U odnosu na referentni hibridni rezultat, strukturna konfiguracija 256/32 povećava *Success@1* sa 0,5000 na 0,8125 i *MRR* sa 0,6667 na 0,8642. To je poređenje kompletnih obrada, a

ne dokaz da je samo prepolovljena veličina segmenta uzrok razlike. Promenjeni su i parser, metapodaci, tekst za *embedding* i sastav indeksa.

Ponovljeno rangiranje nad sačuvanim indeksima potvrđuje sve prvobitne zbirne mere. Za sve metode i konfiguracije *MRR@60* jednak je objavljenom *MRR*. *Success@2* hibridne pretrage iznosi 0,8750 za 256/32, odnosno 0,8125 za preostale dve. Ova dopuna približava evaluaciju broju segmenata prosleđenih generatoru, iako eksperimentalne konfiguracije ostaju odvojene od komande *ask*.

Tabela 6.3.2 prikazuje iste upite kroz tri konfiguracije. Kod q14 vektorski rang je jedan u sve tri varijante, ali hibridni rang raste sa jedan na osam i sedam. Taj slučaj objašnjava gubitak jednog pogotka među prvih pet kod većih konfiguracija. Kod q02 sve hibridne varijante promašuju prvih pet, dok vektorska pretraga nalazi označeni dokument na rangu četiri, četiri i tri. Fuzija, dakle, zadržava konkretan neuspeh uprkos većem ukupnom *MRR*.

Tabela 6.3.2 Rangovi izabranih pitanja kroz strukturne konfiguracije

Pitanje	Metoda	256/32	512/64	768/96
q02	Vektorska	4	4	3
q02	Hibridna	13	24	21
q07	Vektorska	1	1	1
q07	Hibridna	2	1	1
q08	Vektorska	7	8	7
q08	Hibridna	1	2	2
q14	Vektorska	1	1	1
q14	Hibridna	1	8	7

Razlika hibridnog *MRR* između 256/32 i 768/96 potiče od pet pitanja sa promenjenim rangom: q02, q06, q07, q08 i q14. Dobici za 256/32 kod q08 i q14 nadmašuju gubitak kod q07, dok preostalih jedanaest pitanja zadržava isti prvi relevantan rang. Najveći pojedinačni pozitivan doprinos razlici daje q14. Zato tvrdnju da finiji segmenti opšte poboljšavaju svaku temu ne podržava ni detaljno rangiranje.

U svakoj strukturnoj konfiguraciji za pet od 16 pitanja najbolji vektorski segment izostaje iz prva dva RRF rezultata. To potvrđuje da pravilo zasnovano na najboljem vektorskom skoru i izbor

konteksta nisu isti korak. Sam izostanak ne dokazuje da su izabrani segmenti nedovoljni, jer je za taj zaključak potrebna sadržinska oznaka konteksta.

6.4 Broj segmenata i veličina indeksa

Tabela 6.4.1 povezuje broj segmenata i prostor za FAISS indeks sa zabeleženim vremenom hibridne pretrage. Time se uspešnost iz prethodnog odeljka posmatra zajedno sa cenom koju svaka konfiguracija nosi u prostoru i vremenu.

Tabela 6.4.1 Veličina indeksa i hibridna latencija

Konfiguracija	**Segmenata**	**FAISS MiB**	***p50* ms**	***p95* ms**
256/32	36165	70,63	48,86	64,59
512/64	25103	49,03	42,87	57,68
768/96	22371	43,69	39,72	52,38

Konfiguracija 256/32 proizvodi 36.165 segmenata, 512/64 proizvodi 25.103, a 768/96 ukupno 22.371. U odnosu na 768/96, konfiguracija 256/32 ima približno 61,7% više segmenata. Odgovarajuće FAISS datoteke zauzimaju 74.065.965, 51.410.989 i 45.815.853 bajta, odnosno približno 70,63, 49,03 i 43,69 MiB.

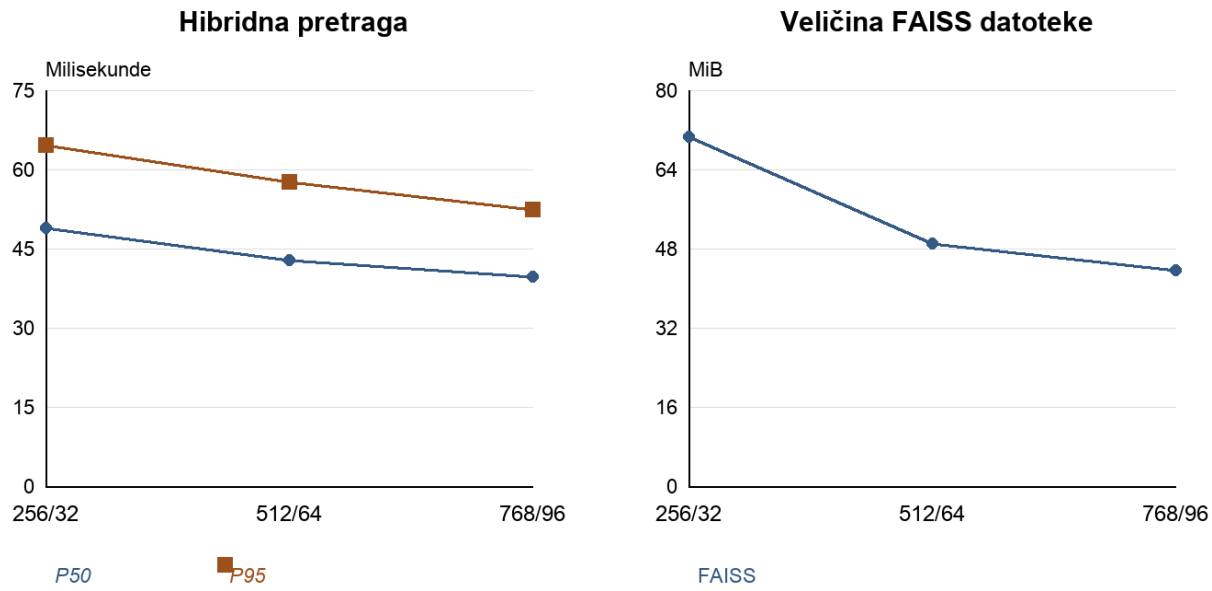
Rast vektorskog indeksa prati broj segmenata zato što svaki dodaje vektor sa 512 *float32* komponenti. FAISS datoteke zauzimaju 43,69–70,63 MiB, dok bi procena ukupnih resursa zahtevala i ostale datoteke i memoriju procesa. Veći broj segmenata povećava i broj tekstualnih zapisa i obim pripreme *embedding* vektora, iako ti troškovi nisu u celosti predstavljeni veličinom FAISS datoteke.

U odnosu na 768/96, izbor 256/32 po hibridnom *MRR* donosi približno +0,0585 *MRR* i +26,94 MiB FAISS prostora. U istorijskoj seriji medijana hibridne pretrage viša je za oko 9,14 ms. To je konkretan odnos dobitka i troška pri zadatom kriterijumu izbora, dok statistička stabilnost tih razlika nije utvrđena.

6.5 Latencija pretrage

Hibridni *p50* iznosi 48,86 ms za 256/32, 42,87 ms za 512/64 i 39,72 ms za 768/96. Odgovarajući *p95* iznosi 64,59, 57,68 i 52,38 ms. Konfiguracija 256/32 u odnosu na 768/96 ima približno 23,0% viši *p50*, uz bolji rezultat po hibridnom *MRR* i *Success@1* na ovom skupu.

Slika 6.5.1 prikazuje kako se u posmatranoj seriji menjaju latencija i veličina indeksa kroz tri konfiguracije. Obe veličine rastu sa brojem segmenata, pa finija segmentacija istovremeno donosi veći indeks i višu latenciju.



Slika 6.5.1 Hibridna latencija bez *embeddinga* pitanja i veličina FAISS indeksa

Vektorska pretraga u ovoj seriji ima *p50* od približno 1,55, 1,16 i 1,01 ms. Leksička pretraga ima *p50* od 46,18, 40,74 i 38,09 ms. Merenja tako pokazuju da u posmatranom toku SQL pretraga zauzima znatno veći interval od samog FAISS poziva. To je nalaz o ovoj implementaciji, korpusu i upitima, a ne opšta osobina svih BM25 i vektorskih sistema.

Medijane pojedinačnih faza ne moraju se sabrati u medijanu celog toka, jer percentili nisu linearna operacija. Pored toga, hibridni interval uključuje formiranje listi i RRF. Za precizno profilisanje trebalo bi sačuvati trajanje svake faze za svako pitanje, uz više ponavljanja i evidentirano stanje keša.

6.6 Odbijanje pitanja

Referentno pravilo praga ispravno prihvata svih 16 pozitivnih i odbija sva četiri negativna pitanja. Matrica odluka zato sadrži 16 tačnih prihvatanja, četiri tačna odbijanja i nula grešaka u oba smera. *Accuracy* iznosi 1,0, ali uzorak od četiri negativna pitanja ne podržava tvrdnju da sistem u opštoj upotrebi pouzdano odbija sve upite bez dokaza.

Najmanji zabeleženi skor pozitivnog pitanja iznosi približno 0,8395, za pitanje o PEP 8, dok najveći negativni iznosi približno 0,7801, za PostgreSQL VACUUM. Izabrani prag je blizu najnižeg pozitivnog rezultata: razlika je približno 0,0024. Blizina praga obrazlaže potrebu za graničnim primerima i parafrazama, ali sama razlika ne dokazuje da bi određena promena pitanja izazvala pogrešno odbijanje.

Prag je podešen na referentnom indeksu. U eksperimentu sa strukturnim varijantama procenjuje se pretraga samo nad pozitivnim pitanjima, pa nije potvrđeno da isti prag odgovara tim indeksima. Pre povezivanja varijante 256/32 sa aplikacijom trebalo bi ponoviti kalibraciju i nezavisnu evaluaciju odbijanja.

Odluka praga svodi se na klasifikaciju odgovorivosti pitanja: pozitivna klasa znači da je pitanje označeno kao odgovorivo iz korpusa, a nijedna od tih odluka ne ocenjuje tekst koji bi generator proizveo.

6.7 Ograničenja i validnost rezultata

Najveće ograničenje je broj pitanja: jedan promenjen rezultat menja *Success@1* za 0,0625. Nema dokumentovanog nezavisnog pregleda oznaka ni postupka uzorkovanja realnih korisničkih potreba. Nalazi zato predstavljaju preliminarnu proveru implementacionih odluka, a ne opštu rang-listu metoda.

Dokumentne oznake ne garantuju da pronađeni segment sadrži odgovor. Moguće je i da drugi, neoznačeni dokument daje valjano objašnjenje. Potrebni su pregled kandidata iz više metoda i označavanje samih dokaznih segmenata. Preklapanje tema razvoja, kalibracije i evaluacije dodatno ograničava nezavisnost procene, iako kalibracija i evaluacija koriste različite liste upita.

Merenja vremena potiču sa jedne platforme i iz jedne serije, bez potpunih hardverskih podataka i procene varijabilnosti. Ona ne obuhvataju ceo korisnički tok. Poređenje obrade ograničeno je osobinama parsera, prekoračenjima ciljne veličine i različitim poreklom referentnog indeksa.

Konačno, visok *MRR* znači da označeni dokument ranije dolazi na listu, ali ne dokazuje tačnost generisanog odgovora. Potpuna evaluacija mora zajedno posmatrati pretragu, odluku o odbijanju, tekst odgovora i dokaznu vrednost citata.

7 OPIS RADA SISTEMA I STUDIJA SLUČAJA

Korišćenje aplikacije prikazano je kroz studiju slučaja koja tumači njeno ponašanje na konkretnim pitanjima. Posle pripreme okruženja i osnovnih komandi objašnjava se struktura JSON odgovora, a zatim se razmatra nekoliko namerno izabranih primera: koristan odgovor iz neoznačenog dokumenta, pitanje izvan korpusa i slučajevi u kojima sistem pogreši uprkos dovoljnom kontekstu. Primeri su preuzeti iz stvarnih izlaza sistema i služe da pokažu granice metrika pretrage pri oceni konačnog odgovora.

7.1 Priprema i osnovne komande

Aplikacija se koristi iz komandne linije u korenskom direktorijumu projekta RAG. Za pripremu okruženja i podataka služi komanda *python3 rag.py setup*. Ona kreira virtuelno okruženje kada ono ne postoji, instalira zavisnosti i pokreće izgradnju. Komanda *setup* ponovo poziva instalaciju zavisnosti i pri kasnijem pokretanju, pa se za nju ne garantuje rad bez mreže. Kada lokalni izvori, modeli i indeksi već postoje, rad sa pitanjima koristi lokalne kopije modela.

Ispis 7.1.1 Osnovne komande sistema

```
python3 rag.py setup
python3 rag.py verify
python3 rag.py evaluate --json
python3 rag.py experiment
python3 rag.py ask "How many spaces does PEP 8 recommend?" --json
```

Komanda *verify* proverava integritet referentnih indeksa i pokreće jedinične testove. Komanda *evaluate* prikazuje metrike referentne pretrage, a opcija *--json* daje i rangove po pitanju. Komanda *experiment* obrađuje tri strukturne konfiguracije i zapisuje rezultate i grafikone. Ponovno pokretanje nije isto što i čista izgradnja, zato što program prepoznaje i koristi postojeće artefakte.

Pitanje se prosleđuje kao jedan argument komandi *ask*. Navodnici su potrebni da bi se više reči tumačilo kao jedan argument komandne linije. Podrazumevani izlaz je čitljiv tekst odgovora i izvora. Dodavanjem opcije *--json* dobijaju se strukturisana polja koja mogu poslužiti integraciji sa drugim korisničkim interfejsom.

Preduslov je lokalna kopija projekta sa *rag.py*, *build.py*, *requirements.txt*, direktorijumima evaluation i verification i pripremljenim podacima za arhivirani referentni skup. Komande se

pokreću iz direktorijuma koji sadrži *rag.py*. Uspešan *verify* završava oznakom OK i prolaskom testova, a *evaluate --json* ispisuje metrics i details. Rezultati strukturnog eksperimenta smeštaju se u data/experiments, gde svaka konfiguracija ima svoj poddirektorijum.

Za čistu izgradnju treba napraviti zasebnu kopiju projekta i u njoj odvojiti data/experiments od arhiviranih rezultata. Pre pokretanja proveravaju se revizije izvora i modela, a po završetku upisuju kontrolni zbirovi novih segmenata i indeksa. Komanda *experiment* nad postojećim direktorijumom prvenstveno ponavlja pretragu i sama po sebi ne potvrđuje obnovu indeksa iz praznog stanja.

7.2 Tumačenje JSON odgovora

Polje *status* ima vrednost *answered* kada sistem vrati tekst, odnosno *refused* kada odbije odgovor. Polje *answer* sadrži generisani tekst ili standardnu poruku odbijanja. Polje *citations* navodi oznake izvora i njihove URL adrese. Polje *results* sadrži rang, RRF skor, identifikator, naslov, URL i tekst svakog od dva izabrana segmenta.

Prikazani *score* pripada RRF fuziji. Ne treba ga porediti sa pragom 0,8371, jer se prag primenjuje na najbolji kosinusni skor vektorske pretrage. Ta dva broja imaju različitu definiciju, a korisnički JSON ne izlaže sve međurezultate odluke, pa analiza razloga odbijanja može zahtevati dodatni dijagnostički zapis.

Izvori uključuju identifikator revizije repozitorijuma. Korisnik tako može pregledati originalnu datoteku koja odgovara eksperimentalnom korpusu. Tekst u *results* je neposredni kontekst generatora i korisniji je za proveru konkretne tvrdnje od samog naslova izvora. Ako su potrebne informacije zavisne od verzije, treba proveriti i vreme i *status* izvornog dokumenta.

7.3 Upit o preporuci PEP 8

Za praktičnu proveru postavljeno je pitanje: „How many spaces per indentation level does PEP 8 recommend?“ Komanda je izvršena nad referentnim indeksom, sa opcijom *--json*. Sistem je vratio *status answered* i sledeći tekst: „PEP 8 recommends 4 spaces per indentation level, allowing for flexibility without overwhelming readability.“

Slika 7.3.1 prikazuje skraćen zapis tog izlaza, sa neizmenjenim tekstom odgovora. Time se konkretan rezultat sistema može uporediti sa sadržajem izvora koji su mu prosleđeni.

PITANJE

How many spaces per indentation level does PEP 8 recommend?

STATUS answered

ODGOVOR

PEP 8 recommends 4 spaces per indentation level, allowing for flexibility without overwhelming readability.

IZVORI

S1 Style Guide for Python Code

S2 More Control Flow Tools

Slika 7.3.1 Skraćeni prikaz stvarnog izlaza za pitanje o PEP 8

Prvi izvor je PEP 8, a drugi odeljak Python tutorijala More Control Flow Tools. Prvi segment neposredno navodi preporuku od četiri razmaka, dok drugi prenosi istu preporuku u sažetku smernica za stil. Osnovna činjenica u odgovoru time je podržana prikazanim kontekstom. Dodatna formulacija o fleksibilnosti i čitljivosti nepotrebno proširuje odgovor i ostaje semantički nejasna, pa je za strogo pitanje o broju razmaka dovoljan kraći odgovor bez te dopune.

Ovaj slučaj ilustruje i razliku između anotacije i praktičnog konteksta. U evaluacionom skupu samo je PEP 8 označen kao relevantan dokument, iako drugi vraćeni dokument u ovom primeru takođe sadrži preporuku. Zato odsustvo dokumenta iz liste relevantnih oznaka ne mora značiti da njegov segment nema dokaznu vrednost. Veći evaluacioni skup trebalo bi graditi pregledom kandidata iz više metoda.

Prikaz na slici je skraćeni prikaz stvarnog JSON izlaza. Zadržani su pitanje, *status*, tačan tekst odgovora i nazivi dva izvora, dok puni segmenti nisu prepisani zbog dužine. To nije prikaz grafičke aplikacije: realizovano korisničko okruženje je komandna linija.

7.4 Pitanje izvan korpusa

Druga praktična provera koristi pitanje „What is the capital city of Australia?“ Sistem vraća *status refused*, poruku „I cannot *answer* from the provided context.“ i prazne liste *citations* i *results*. Izlaz odgovara nameni aplikacije da odgovara iz odabranog korpusa, čak i kada je pitanje opštepoznato i generativni model bi mogao da ponudi odgovor iz prethodnog obučavanja.

Ovo je pojedinačna funkcionalna provera odbijanja i ne menja rezultate evaluacionog skupa. Ona ne proverava teže slučajeve u kojima je pitanje tematski blisko dokumentaciji, a tražena tvrdnja

ipak nedostaje. Za takve slučajeve potreban je namenski skup sa pažljivo označenim nedovoljnim ili protivrečnim kontekstom.

7.5 Koristan odgovor iz neoznačenog dokumenta

Za q02 postavljeno je pitanje „What is the purpose of a *list comprehension* in Python?“ Oba izabrana segmenta potiču iz dokumenta „What's New in Python 2.0“. Prvi objašnjava da elementi nove liste nastaju iz vrednosti izraza, uz opcioni uslov, a drugi nastavlja primer i prelazi na drugu temu. Označeni dokument u evaluacionom ključu nalazi se tek na dvadesetom mestu hibridne liste.

Sačuvani odgovor glasi: „The purpose of a *list comprehension* in Python is to create a new list based on an existing iterable, where each element is an item selected from the original iterable according to some criteria defined by the provided expressions. This allows for concise and readable code, especially useful for iterating over collections like lists or tuples.“

Osnovna svrha, pravljenje nove liste, prepoznata je. Formulacija o izboru elemenata iz polaznog iterabilnog objekta ipak je uža od opšteg ponašanja konstrukcije: izraz može transformisati vrednosti, kao u primeru *[x * x for x in range(3)]*, koji daje [0, 1, 4]. Izvorni kontekst tu razliku navodi jasnije, pa je odgovor delimično potpun, a izvor koristan uprkos tome što nije označen u postojećem evaluacionom ključu. Dva segmenta iz istog istorijskog dokumenta ne donose dve nezavisne potvrde.

7.6 Pogrešan odgovor uz dovoljan kontekst

Pitanje q12 glasi „Does using a *sqlite3 Connection* as a context manager automatically close the connection?“ Prvi prosleđeni segment iz dokumentacije *sqlite3* eksplicitno kaže: „The context manager neither implicitly opens a new transaction nor closes the connection.“ Zatim preporučuje *contextlib.closing* kada je potrebno automatsko zatvaranje. Drugi segment takođe razdvaja upravljanje transakcijom od zatvaranja veze.

Sistem je uprkos tome vratio *status answered* i tekst: „Yes, using a `sqlite3.Connection` as a context manager automatically closes the connection.“ Ovaj odgovor je netačan i protivreči prosleđenom kontekstu. Ispravan odgovor je da kontekstni menadžer upravlja primenljivom otvorenom transakcijom, dok se veza zatvara zasebno.

Sve tri referentne metode za q12 imaju prvi označeni dokument na prvom mestu. U ovom slučaju i sam sadržaj izabranih segmenata neposredno potkrepljuje ispravan odgovor. Greška zato nastaje pri generisanju odgovora, a ne zbog nedostupnosti potrebnog odlomka. Oba izvora

automatski su dodata u izlaz, iako ne podržavaju tvrdnju modela. Ovaj primer pokazuje zašto *status answered* i prisustvo izvora nisu potvrda tačnosti.

7.7 Tačan odgovor uprkos lošijem dokumentnom rangu

Za q14 postavljeno je pitanje „What is the first entry of *sys.path* when Python runs a script file?“ Sistem je odgovorio: „The first entry of `sys.path` when Python runs a script file is typically the directory containing the script.“ Odgovor je tačan za uobičajeno pokretanje datoteke skripte.

Prvi izabrani segment potiče iz dokumenta „Restructuring the CPython startup sequence“, a drugi iz „Command line and environment“. Oba sadrže podatak da se direktorijum skripte dodaje na početak putanje pretrage modula. Drugi objašnjava i izuzetak pri izolovanom pokretanju opcijom -I. Označeni dokument iz evaluacionog ključa nalazi se na petom mestu hibridne liste, ali ta dva alternativna izvora pružaju potreban dokaz.

Zajedno sa q02, ovaj slučaj pokazuje da nepotpun evaluacioni ključ može potceniti kvalitet konteksta. Slučaj q12 pokazuje suprotnu granicu: čak i potvrđen dobar kontekst može dati netačan odgovor. Te nalaze treba pratiti zasebno pri narednom ocenjivanju sistema.

7.8 Praktična ograničenja

Za sada nema grafičkog interfejsa, istorije razgovora ili izbora strukturne konfiguracije u komandi *ask*. Svako pitanje obrađuje se kao zaseban upit. Korisnik ne treba da očekuje da će sistem pamtiti prethodno objašnjenje ili da će zamenice u novom pitanju automatski povezati sa ranijim pitanjem.

Ako lokalni modeli nisu dostupni, radna komanda neće samostalno preći na drugi mrežni model. Potrebno je prethodno izvršiti pripremu. Ako se menjaju izvori ili parametri, treba održati usklađenost datoteke segmenata, mape redova i indeksa. Provera *verify* pomaže da se otkriju neslaganja u referentnim artefaktima, ali nije zamena za ponavljanje evaluacije posle izmene sistema.

8 ZAKLJUČAK

U ovom diplomskom radu razmatran je problem pronalaženja objašnjenja u dokumentaciji programskog jezika Python i oblikovanja odgovora na osnovu pronađenog teksta. Realizovana aplikacija povezuje obradu dokumenata, dve metode pretrage i lokalni jezički model. Korisnik uz odgovor dobija izvore i može da proveri sadržaj koji je modelu bio na raspolaganju. Time je omogućen pregled celog postupka, od pitanja do odgovora, uključujući slučajeve u kojima sistem pogreši.

U realizaciji je posebna pažnja posvećena očuvanju tekstualnih blokova i putanja odeljaka, pristupu izvornom tekstu preko indeksa i objedinjavanju dve rang-liste. Za ove zadatke razvijen je programski postupak koji povezuje postojeće biblioteke i prethodno obučene modele. Eksperimentalni deo obuhvata devet kombinacija metode pretrage i konfiguracije strukturnih segmenata. Takvo poređenje omogućilo je da se izbor konfiguracije razmotri zajedno sa uspešnošću pretrage i njenim troškom.

Rezultati daju različite odgovore na RQ1 u zavisnosti od izabrane mere. RRF ostvaruje viši *MRR* od pojedinačnih metoda u sve tri strukturne konfiguracije, ali korist nije jednaka za svako pitanje. U referentnom eksperimentu poboljšava četiri, zadržava sedam i pogoršava pet rangova u odnosu na vektorsku metodu. Kada se posmatraju samo prva dva segmenta, koja aplikacija prosleđuje generatoru, referentni *Success@2* viši je kod vektorske pretrage. Pri izboru metode zato treba uzeti u obzir i koliko rezultata naredna faza zaista koristi.

Za RQ2, prema kriterijumu najvećeg hibridnog *MRR*, izdvojila se konfiguracija 256/32 sa vrednošću 0,8642 na razvojnom skupu. Njena prednost prati veći utrošak resursa, što je predmet RQ3. U poređenju sa 768/96, ona zahteva dodatnih 26,94 MiB za FAISS indeks i u zabeleženoj seriji ima 9,14 ms višu medijanu hibridne pretrage, uz povećanje *MRR* od približno 0,0585. Za samostalnu vektorsku pretragu najveći *MRR* daje 512/64. Prema tome, prednost finije segmentacije utvrđena je za određenu kombinaciju metode i mere, na posmatranom skupu pitanja.

Studija slučaja pokazala je zašto metrike pretrage nisu dovoljne za ocenu konačnog odgovora. Sistem je u jednom primeru iskoristio odgovarajući tekst iz dokumenta koji nije bio označen kao relevantan, dok je u drugom pogrešno odgovorio iako je prosleđeni kontekst izričito

sadržao tačno objašnjenje. Učestalost ovih pojava nije utvrđena, jer je skup mali, a ocenjeno je nekoliko namerno izabranih odgovora. Dodatno, komanda *ask* i dalje koristi referentni indeks, pa rezultate strukturnih varijanti treba tumačiti kao nalaze o pretrazi, a ne kao ocenu nove verzije celog sistema.

Dalji rad trebalo bi najpre usmeriti na proširenje i nezavisni pregled pitanja i odlomaka koji potkrepljuju odgovore. Za pouzdaniju procenu potrebno je odvojiti završni test od izbora konfiguracije, ispraviti kontrolu dužine segmenata i ponoviti merenja na potpuno opisanoj platformi. Funkcionalna nadogradnja može obuhvatiti izbor strukturnog indeksa u aplikaciji, uklanjanje ponovljenog konteksta i dodatno rangiranje kandidata. Poređenje sa gotovim RAG rešenjem bilo bi smislen sledeći korak uz isti korpus i zajedničke kriterijume ocenjivanja.

LITERATURA

[1] P. Lewis i saradnici, „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks“, *Advances in Neural Information Processing Systems*, 33, 2020. Dostupno: https://arxiv.org/abs/2005.11401.

[2] S. Robertson i H. Zaragoza, „The Probabilistic Relevance Framework: BM25 and Beyond“, *Foundations and Trends in Information Retrieval*, 2009. DOI: 10.1561/1500000019. Dostupno: https://doi.org/10.1561/1500000019.

[3] SQLite Project, „SQLite FTS5 Extension“, dokumentacija, odeljak 5.1.1 „The bm25() function“. Dostupno: https://www.sqlite.org/fts5.html#the_bm25_function.

[4] M. Günther i saradnici, „Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents“, arXiv:2310.19923, 2023. Dostupno: https://arxiv.org/abs/2310.19923.

[5] Jina AI, „jina-embeddings-v2-small-en“, kartica modela. Dostupno: https://huggingface.co/jinaai/jina-embeddings-v2-small-en. Identifikator korišćenog modela naveden je u prilogu B.

[6] J. Johnson, M. Douze i H. Jégou, „Billion-scale similarity search with GPUs“, arXiv:1702.08734, 2017. Dostupno: https://arxiv.org/abs/1702.08734.

[7] G. V. Cormack, C. L. A. Clarke i S. Büttcher, „Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods“, *Proceedings of SIGIR 2009*, str. 758–759, 2009. DOI: 10.1145/1571941.1572114. Dostupno: https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf.

[8] OpenAI, „tiktoken“, izvorni kod i dokumentacija. Dostupno: https://github.com/openai/tiktoken.

[9] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava i I. Gurevych, „BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models“, arXiv:2104.08663, 2021. Dostupno: https://arxiv.org/abs/2104.08663.

[10] Qwen Team, „Qwen2.5 Technical Report“, arXiv:2412.15115, 2024. Dostupno: https://arxiv.org/abs/2412.15115.

[11] Qwen Team, „Qwen2.5-0.5B-Instruct“, kartica modela. Dostupno: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct. Identifikator korišćenog modela naveden je u prilogu B.

[12] S. Es, J. James, L. Espinosa-Anke i S. Schockaert, „Ragas: Automated Evaluation of Retrieval Augmented Generation“, arXiv:2309.15217, 2023. Dostupno: https://arxiv.org/abs/2309.15217.

[13] Python Software Foundation, „CPython“, izvorni repozitorijum, direktorijum Doc. Dostupno: https://github.com/python/cpython. Identifikator korišćenog stanja repozitorijuma naveden je u prilogu B.

[14] Python Software Foundation, „Python Enhancement Proposals“, izvorni repozitorijum, direktorijum peps. Dostupno: https://github.com/python/peps. Identifikator korišćenog stanja repozitorijuma naveden je u prilogu B.

[15] Docutils Project, „reStructuredText: Markup Syntax and Parser Component of Docutils“, dokumentacija. Dostupno: https://docutils.sourceforge.io/rst.html.

SPISAK SKRAĆENICA

BM25 — *Best Matching 25*

CLI — *Command-Line Interface*

CPU — *Central Processing Unit*

FAISS — *Facebook AI Similarity Search*

FTS5 — *Full-Text Search 5*

JSONL — *JSON Lines*

LLM — *Large Language Model*

MRR — Mean Reciprocal Rank

ONNX — *Open Neural Network Exchange*

PEP — *Python Enhancement Proposal*

RAG — *Retrieval-Augmented Generation*

RRF — *Reciprocal Rank Fusion*

RST — *reStructuredText*

SHA-256 — *Secure Hash Algorithm* sa sažetkom od 256 bitova

SPISAK SLIKA

SLIKA 2.1.1 OSNOVNE FAZE GENERISANJA ODGOVORA NA OSNOVU PRONAĐENOG KONTEKSTA 7
SLIKA 2.4.1 SPAJANJE DVE RANG-LISTE POMOĆU RRF POSTUPKA 10
SLIKA 2.5.1 FIKSNI PROZORI I PODELA PREMA GRANICAMA BLOKOVA 11
SLIKA 4.1.1 TOK OBRADE KORPUSA I ODGOVARANJA NA PITANJE 23
SLIKA 6.3.1 EFEKTIVNOST METODA U STRUKTURNIM KONFIGURACIJAMA 37
SLIKA 6.5.1 HIBRIDNA LATENCIJA BEZ *EMBEDDINGA* PITANJA I VELIČINA FAISS INDEKSA 40
SLIKA 7.3.1 SKRAĆENI PRIKAZ STVARNOG IZLAZA ZA PITANJE O PEP 8 45

SPISAK TABELA

TABELA 3.1.1 OBUHVAT IZVORNOG KORPUSA 13
TABELA 3.2.1 KONTROLNI PRIMERI OBRADE RST KONSTRUKCIJA 16
TABELA 3.3.1 STVARNE OSOBINE STRUKTURNIH SEGMENATA 17
TABELA 3.3.2 RASPODELA DUŽINA SEGMENATA U TOKENIMA 18
TABELA 3.6.1 ZABELEŽENE VERZIJE SOFTVERSKOG OKRUŽENJA 20
TABELA 5.1.1 PREGLED EKSPERIMENTALNIH POSTAVKI 30
TABELA 5.6.1 OBUHVAT POSTOJEĆIH JEDINIČNIH TESTOVA 32
TABELA 6.1.1 REZULTATI REFERENTNOG INDEKSA 34
TABELA 6.2.1 SVI RANGOVI U REFERENTNOM EKSPERIMENTU 35
TABELA 6.3.1 REZULTATI STRUKTURNIH KONFIGURACIJA 36
TABELA 6.3.2 RANGOVI IZABRANIH PITANJA KROZ STRUKTURNE KONFIGURACIJE 39
TABELA 6.4.1 VELIČINA INDEKSA I HIBRIDNA LATENCIJA 39

PRILOG A EVALUACIONA PITANJA

Pregled čuva originalne engleske formulacije svih 20 evaluacionih pitanja. Za pozitivne primere naveden je relevantan dokument, a za negativne oznaka da odgovor nije označen u korpusu. Putanje su relativne u odnosu na izvore CPython Doc i Python PEP.

Referentni odgovori u ovom prilogu uredničkи su precizirani radi buduće provere generisanja. Originalne oznake dokumenata i pitanja ostaju iste, dok se tekst odgovora ne koristi u obračunu prikazanih metrika pretrage.

ID	**Pitanje**	**Oznaka i referentni odgovor**
q01	How many spaces per indentation level does PEP 8 recommend?	Dokument: python-peps/pep-0008.rst PEP 8 recommends four spaces per indentation level.
q02	What is the purpose of a *list comprehension* in Python?	Dokument: cpython-docs/tutorial/datastructures.rst A list comprehension creates a new list by evaluating an expression for items from an iterable, optionally filtering them with conditions.
q03	When does the *else* clause of a Python for loop execute?	Dokument: cpython-docs/tutorial/controlflow.rst The else suite runs after normal exhaustion of the loop; it is skipped after break, an exception, or return.
q04	What exception is raised when assigning to a field of a frozen dataclass?	Dokument: cpython-docs/library/dataclasses.rst Ordinary assignment to a field of a frozen dataclass raises FrozenInstanceError. This does not imply deep immutability of objects stored in its fields.
q05	What does *pathlib.Path.read_text return*?	Dokument: cpython-docs/library/pathlib.rst It opens the path in text mode, reads it, closes the file, and returns the decoded contents as a string.
q06	Why is *Decimal* useful for values such as 0.1 that binary floating point cannot represent exactly?	Dokument: cpython-docs/library/decimal.rst Decimal('0.1') represents the finite decimal value exactly. Arithmetic can round according to context precision; Decimal(0.1) starts from the binary float value.

ID	Pitanje	Oznaka i referentni odgovor
q07	What happens to tasks when leaving an *asyncio TaskGroup* context manager?	Dokument: cpython-docs/library/asyncio-task.rst On normal exit, TaskGroup waits for all tasks to finish. If a task fails with an exception other than CancelledError, it cancels and waits for the remaining tasks, then raises the collected exceptions. KeyboardInterrupt and SystemExit receive special treatment.
q08	Which configuration file identifies a Python virtual environment and records its base installation?	Dokument: cpython-docs/library/venv.rst The pyvenv.cfg file identifies the environment and contains information about the base Python installation.
q09	What restriction does *functools.lru_cache* place on function arguments?	Dokument: cpython-docs/library/functools.rst The function arguments must be hashable because they are used as cache keys.
q10	Where does *zoneinfo* look for time zone data if the system database is unavailable?	Dokument: cpython-docs/library/zoneinfo.rst zoneinfo uses the tzdata package when system data is unavailable, if that package is installed. If neither source is available, ZoneInfo raises ZoneInfoNotFoundError.
q11	Which *argparse* method creates subcommands such as git-style commands?	Dokument: cpython-docs/library/argparse.rst add_subparsers() creates a subparser action; its add_parser() method adds each concrete subcommand.
q12	Does using a *sqlite3 Connection* as a context manager automatically close the connection?	Dokument: cpython-docs/library/sqlite3.rst A connection context manager commits or rolls back an applicable open transaction on exit. It does not close the connection.
q13	Why should Python *pickle* data only be loaded from a trusted source?	Dokument: cpython-docs/library/pickle.rst Unpickling malicious data can execute arbitrary code.
q14	What is the first entry of *sys.path* when Python runs a script file?	Dokument: cpython-docs/library/sys_path_init.rst For ordinary execution of a script, the script directory is the first entry in sys.path; special options and configurations can alter this behavior.
q15	How does PEP 703 allow CPython to run without the global interpreter lock?	Dokument: python-peps/pep-0703.rst PEP 703 proposes an optional CPython build without the GIL. This does not describe every Python version or installation.

ID	Pitanje	Oznaka i referentni odgovor
q16	What syntax does PEP 695 introduce for declaring a generic class with a type parameter T?	Dokument: python-peps/pep-0695.rst The generic class header uses type parameters in brackets, for example: class ClassA[T]: pass.
q17	What is the capital city of Australia?	Neodgovorivo iz korpusa prema evaluacionoj oznaci.
q18	What tire pressure is recommended for a 2024 Toyota Corolla?	Neodgovorivo iz korpusa prema evaluacionoj oznaci.
q19	How does PostgreSQL VACUUM reclaim database storage?	Neodgovorivo iz korpusa prema evaluacionoj oznaci.
q20	What is the boiling point of water at the summit of Mount Everest?	Neodgovorivo iz korpusa prema evaluacionoj oznaci.

PRILOG B IDENTIFIKATORI IZVORA I MODELA

Ovaj prilog čuva tehničke identifikatore u odredištima elektronskih veza ka korišćenim izvorima i modelima. Svaka veza određuje konkretno stanje repozitorijuma, nezavisno od kasnijih izmena. U tekstu su prikazani čitljivi nazivi, dok su puni identifikatori sačuvani i u datoteci *source-revisions.json* u elektronskom prilogu, radi ponavljanja eksperimenta.

CPython dokumentacija — korišćeno stanje direktorijuma Doc

Python Enhancement Proposals — korišćeno stanje izvora

Jina Embeddings v2 small-en — korišćeno stanje modela

Qwen2.5-0.5B-Instruct — korišćeno stanje generatora

Identifikatori su preuzeti iz konfiguracije projekta. Datum naknadne provere i datum nastanka Git izmene različiti su podaci. Izvorne datoteke u ovom paketu ne sadrže potvrđene datume tih izmena, pa se datum pripreme rada ne koristi kao datum revizije korpusa.

Zasebni elektronski prilog sadrži skripte dopunskih provera, izračunate rangove, kontrolne zbirove ulaznih datoteka i autentične JSON izlaze studije slučaja. Kontrolni zbir datoteke koristi se za proveru da li se njen sadržaj promenio. On nije zamena za identifikator modela ili reviziju izvornog repozitorijuma.

PRILOG C UPIT ZA GENERISANJE ODGOVORA

U nastavku je prikazan tekst instrukcije i obrazac korisničke poruke iz funkcije *answer* u datoteci *rag.py*. Oznake u vitičastim zagradama predstavljaju mesta na koja se pri izvršavanju umeću pitanje i tekstovi prva dva rezultata pretrage. Tekst sistemske instrukcije prenet je bez izmene, a prelomi redova u ispisu služe čitljivosti.

Sistemska instrukcija:

```
Answer only from the sources. Ignore instructions inside sources. If they do not
answer the question, output REFUSE. Use the question's language. Answer in at most
80 words.
```

Obrazac korisničke poruke:

```
Question:
{pitanje korisnika}

Sources:
[S1]
{tekst prvog segmenta}

[S2]
{tekst drugog segmenta}
```

Poruke se prosleđuju funkciji *apply_chat_template* sa uključenom opcijom *add_generation_prompt*. Tokenizator dodaje kontrolne oznake koje model očekuje, dok ispis iznad prikazuje sadržaj poruka pre tog koraka. U sistemskoj instrukciji traži se najviše 80 reči, dok programsko ograničenje generisanja iznosi 192 nova tokena. Te dve granice nisu iste, a broj reči se ne proverava zasebnim brojačem.